Validating Predictions

Last modified: 2026-10-15 00:59:48 (PDT)

Data sets, input matrices and outcome vectors

Definition (Data set)

Definition 0.1 (Data set)**.** A **data set** is a finite collection of n labelled observations, that is, n pairs $(\tilde{x}_i, y_i)$, where $\tilde{x}_i$ is an input vector and y_i is the corresponding outcome:

$$\mathcal{D} \stackrel{\text{def}}{=} \{(\tilde{x}_i, y_i) : i = 1, \ldots, n\}. \tag{1}$$

Remark (The sets we split a data set into)

Remark 0.1 (The sets we split a data set into). We divide a data set into subsets, each with its own job. "Set" in the names training set, validation set and test set is short for "subset of the data set". A training/validation/test split assigns each observation to one of them, and the training set is the subset a fitting procedure is applied to.

Definition (Input matrix)

Definition 0.2 (Input matrix)**.** The **input matrix X** of a data set $\mathcal{D}$ stacks the n inputs $\tilde{x}_i$ as rows, with one column for each of the p input variables:

$$\mathbf{X} = \begin{bmatrix} \tilde{x}_1^\top \\ \vdots \\ \tilde{x}_n^\top \end{bmatrix}. \tag{2}$$

Definition (Outcome vector)

Definition 0.3 (Outcome vector)**.** The **outcome vector** $\tilde{y}$ of a data set $\mathcal{D}$ stacks its n outcomes y_i:

$$\tilde{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}. \tag{3}$$

Example (The input matrix and outcome vector of the first five cars)

Example 0.1 (The input matrix and outcome vector of the first five cars)**.** In the `mtcars` data frame, take the fuel economy `mpg` as the outcome y and the gross horsepower `hp` and weight `wt` as the $p = 2$ inputs. The first five cars give a data set of $n = 5$ observations.

```
# take the first five cars from the mtcars data frame
cars5 <- head(mtcars, 5)

# stack the two inputs as columns, one row per car
X_cars5 <- as.matrix(cars5[, c("hp", "wt")])

# stack the outcomes into one vector
y_cars5 <- cars5$mpg

# show the input matrix and the outcome vector side by side
cbind(X_cars5, mpg = y_cars5)
#>                   hp    wt  mpg
#> Mazda RX4         110 2.620 21.0
#> Mazda RX4 Wag     110 2.875 21.0
#> Datsun 710         93 2.320 22.8
#> Hornet 4 Drive    110 3.215 21.4
#> Hornet Sportabout 175 3.440 18.7
```

The input matrix `X_cars5` has 5 rows, one per car, and 2 columns, one per input variable. The outcome vector `y_cars5` has 5 entries, one per car.

Fitting and scoring prediction rules

Definition (Fitting procedure)

Definition 0.4 (Fitting procedure)**.** A **fitting procedure** is a rule g that, given any data $((x_i, y_i))_{i \in I}$ and any covariate value x, computes a prediction of the outcome at x:

$$(x; ((x_i, y_i))_{i \in I}) \mapsto g(x; ((x_i, y_i))_{i \in I}). \tag{4}$$

Source

The rule g of the definition of a prediction. Hastie et al. (2009, sec. 7.2) write the model estimated from a training set as $\hat{f}$.

Definition (Training set)

Definition 0.5 (Training set)**.** A **training set** is the data to which a fitting procedure is applied: n observations, written as an ordered list indexed by a set I of n observation numbers:

$$\mathcal{D}_{\text{train}} \stackrel{\text{def}}{=} ((\tilde{x}_i, y_i))_{i \in I}, \quad |I| = n. \tag{5}$$

Source

Hastie et al. (2009, sec. 7.2), who also write the training set as $\mathcal{T}$.

Definition (Prediction rule)

Definition 0.6 (Prediction rule)**.** The **prediction rule** $\hat{y}$ that a fitting procedure g produces from a training set $\mathcal{D}_{\text{train}}$ is the function that maps each covariate value x to the prediction that g computes from x and $\mathcal{D}_{\text{train}}$:

$$\hat{y} : x \mapsto g(x; \mathcal{D}_{\text{train}}). \tag{6}$$

Definition (Generalization error)

Definition 0.7 (Generalization error)**.** The **generalization error** of a prediction rule $\hat{y}$ fitted to a training set $\mathcal{D}_{\text{train}}$ is its expected squared prediction error on a new observation (X_*, Y_*) drawn from the same population independently of $\mathcal{D}_{\text{train}}$, with $\mathcal{D}_{\text{train}}$ held fixed:

$$\text{Err}_{\mathcal{D}_{\text{train}}} \stackrel{\text{def}}{=} \text{E}\left[(\hat{y}(X_*) - Y_*)^2 \mid \mathcal{D}_{\text{train}}\right]. \tag{7}$$

Source

The risk form follows Brian Hutchinson's Fall 2025 lecture notes; the conditional form, with the training set held fixed, follows Hastie et al. (2009, sec. 7.2).

Definition (Held-out set)

Definition 0.8 (Held-out set)**.** A **held-out set** for a training set $\mathcal{D}_{\text{train}}$ is a list of m further observations, none of them in $\mathcal{D}_{\text{train}}$, that the fitting procedure does not use and on which the prediction rule $\hat{y}$ is scored:

$$\big((x_{*,1}, y_{*,1}), \dots, (x_{*,m}, y_{*,m})\big). \tag{8}$$

Source

James et al. (2021, sec. 5.1.1), who call it a hold-out set.

Definition (Root mean squared error of predictions)

Definition 0.9 (Root mean squared error of predictions)**.** The **root mean squared error** (RMSE) of predictions $\hat{y}_1, \dots, \hat{y}_n$ of observed outcomes $y_1, \dots, y_n$ is the square root of their mean squared error:

$$\text{RMSE}(\hat{y}) \stackrel{\text{def}}{=} \sqrt{\text{MSE}(\hat{y})}. \tag{9}$$

Definition (Training error)

Definition 0.10 (Training error)**.** The **training error** of a prediction rule $\hat{y}$ fitted to a training set $\mathcal{D}_{\text{train}}$ is the mean squared error of its fitted values $\hat{y}_i$, $i \in I$, as predictions of the training outcomes y_i:

$$\overline{\text{err}} \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i \in I} (\hat{y}_i - y_i)^2. \tag{10}$$

Source

Hastie et al. (2009, sec. 7.2), who write the training error as $\overline{\text{err}}$ and define it for any loss.

Overfitting

Exercise (Training error versus error on new cars)

Exercise 0.1 (Training error versus error on new cars)**.** The built-in `mtcars` data frame has $n = 32$ cars. Fit four models for fuel efficiency `mpg` as a polynomial in weight `wt`, of degree 1, 2, 3, and 4, using only the first 20 rows as the training set. For each model, compute the RMSE of its predictions on the 20 training rows and on the remaining 12 rows, a held-out set.

1. Which degree has the smallest training RMSE?
2. Which degree has the smallest RMSE on the 12 held-out rows?
3. Do the terms added beyond degree 2 improve predictions for the held-out cars?

Solution

Solution 0.1.

```r
train_cars <- mtcars[1:20, ]
held_out_cars <- mtcars[21:32, ]

prediction_mse <- function(model, data) {
  response_only <- stats::update(stats::formula(model), . ~ 1)
  y <- stats::model.response(stats::model.frame(response_only, data))
  mean((predict(model, newdata = data) - y)^2)
}

rmse <- function(model, data) {
  sqrt(prediction_mse(model, data))
}

overfitting_results <- tibble::tibble(degree = 1:4) |>
  dplyr::mutate(
    fit = purrr::map(
      degree,
      \(d) lm(mpg ~ poly(wt, d, raw = TRUE), data = train_cars)
    ),
    training_RMSE = purrr::map_dbl(fit, rmse, data = train_cars),
    held_out_RMSE = purrr::map_dbl(fit, rmse, data = held_out_cars)
  ) |>
  dplyr::select(-fit)

overfitting_results
#> # A tibble: 4 x 3
#>   degree training_RMSE held_out_RMSE
#>    <int>         <dbl>         <dbl>
#> 1      1          2.91          3.45
#> 2      2          2.46          3.18
#> 3      3          2.45          3.21
#> 4      4          2.44          3.19
```

```r
best_train <- overfitting_results$degree[
  which.min(overfitting_results$training_RMSE)
]
best_held_out <- overfitting_results$degree[
  which.min(overfitting_results$held_out_RMSE)
]

c(best_train = best_train, best_held_out = best_held_out)
#>    best_train best_held_out
#>             4             2
```

1. The smallest training RMSE is at degree 4. Each model contains the previous one as a special case, so its training RMSE cannot increase as the degree grows.
2. The smallest held-out RMSE is at degree 2.
3. No. Going from degree 2 to degree 3 or 4 lowers the training RMSE a little but leaves the held-out RMSE essentially unchanged (it does not fall below its degree-2 value). The terms beyond degree 2 improve the fit to the 20 training cars without improving predictions for the 12 held-out cars. Degree 1 to degree 2 is different: there the held-out RMSE also falls.

Definition (Overfitting)

Definition 0.11 (Overfitting)**. Overfitting** occurs when a prediction rule fits its training set well but predicts poorly for new observations.

Example (Overfitting in polynomial fits to cars)

Example 0.2 (Overfitting in polynomial fits to cars)**.** In Exercise 0.1, the terms beyond degree 2 fit the 20 training cars more closely but do not predict the 12 held-out cars better. The effect is small here: it is the beginning of overfitting, not a dramatic case.

Generalization error

Exercise (Score a fitted model on a car it has not seen)

Exercise 0.2 (Score a fitted model on a car it has not seen)**.** A prediction rule $\hat{y}$ has been fitted to a training set $\mathcal{D}_{\text{train}}$ of n observations, such as the 20 training cars of Exercise 0.1. A new car (X_*, Y_*) is then drawn from the same population, independently of $\mathcal{D}_{\text{train}}$.

1. Holding $\mathcal{D}_{\text{train}}$ fixed, write the average squared prediction error that $\hat{y}$ would make over all such new cars, as an expectation.
2. Which parts of your expression do you know, and which do you not know?
3. The training RMSE of Exercise 0.1 is the square root of the training error $\overline{\text{err}}$. What does $\overline{\text{err}}$ use in place of your expectation?

Solution

Solution 0.2.

1. Average the squared prediction error over the distribution of (X_*, Y_*), with $\mathcal{D}_{\text{train}}$, and so the rule $\hat{y}$, held fixed:

$$\mathrm{E}\left[(\hat{y}(X_*) - Y_*)^2 \mid \mathcal{D}_{\text{train}}\right].$$

2. The rule $\hat{y}$ is known, because we fitted it. The joint distribution of (X_*, Y_*) is unknown, so the expectation cannot be computed. If we knew that distribution, we would not need data to build a prediction rule.

3. The training error $\overline{\text{err}}$ replaces the expectation over new cars with an average over the n training cars. Those cars are draws from the same population, but they are not independent of $\mathcal{D}_{\text{train}}$: they *are* $\mathcal{D}_{\text{train}}$, and $\hat{y}$ was chosen to fit them.

Definition (Expected generalization error)

Definition 0.12 (Expected generalization error)**.** The **expected generalization error** of a fitting procedure g at training-set size n is the mean, over training sets $\mathcal{D}_{\text{train}}$ of n observations drawn independently from the population, of the generalization error of the prediction rule that g produces from $\mathcal{D}_{\text{train}}$:

$$\mathrm{Err}^g(n) \stackrel{\text{def}}{=} \mathrm{E}\left[\mathrm{Err}_{\mathcal{D}_{\text{train}}}\right], \quad \mathcal{D}_{\text{train}} = ((X_i, Y_i))_{i \in I}, \quad |I| = n. \tag{11}$$

Source

Hastie et al. (2009, sec. 7.2) call this quantity the *expected prediction error* (or *expected test error*) and write it Err, without the argument n.

Definition (Mean function)

Definition 0.13 (Mean function)**.** The **mean function** μ of an outcome Y given a covariate X maps each covariate value x to the mean of Y given $X = x$:

$$\mu(x) \stackrel{\text{def}}{=} \mathrm{E}[Y \mid X = x]. \tag{12}$$

Definition (Expected squared prediction error at a covariate value)

Definition 0.14 (Expected squared prediction error at a covariate value)**.** The **expected squared prediction error at** x of a fitting procedure g at training-set size n is the mean squared prediction error of its prediction at x for a new outcome Y_* with $X_* = x$, averaged over random training sets $\mathcal{D}_{\text{train}}$ of n observations drawn independently of (X_*, Y_*), and over Y_*:

$$\mathrm{EPE}_n^g(x) \stackrel{\text{def}}{=} \mathrm{E}\Big[\big(g(x; \mathcal{D}_{\text{train}}) - Y_*\big)^2 \mid X_* = x\Big]. \tag{13}$$

Source

Hastie et al. (2009, sec. 7.3), who write this quantity as $\mathrm{Err}(x_0)$.

Exercise (The expected generalization error, covariate value by covariate value)

Exercise 0.3 (The expected generalization error, covariate value by covariate value)**.** Let g be a fitting procedure, and let the random training set $\mathcal{D}_{\text{train}}$ of n observations be independent of the new observation (X_*, Y_*). Show that the expected generalization error is the mean of the expected squared prediction error at x over the distribution of X_*:

$$\mathrm{Err}^g(n) = \mathrm{E}[\mathrm{EPE}_n^g(X_*)].$$

Solution

Solution 0.3.

$$
\begin{aligned}
&\mathrm{Err}^g(n) \\
&\quad = \mathrm{E}\left[\mathrm{Err}_{\mathcal{D}_{\text{train}}}\right] && \text{(definition)} \\
&\quad = \mathrm{E}\left[\mathrm{E}\left[(\hat{y}(X_*) - Y_*)^2 \mid \mathcal{D}_{\text{train}}\right]\right] && \text{(definition)} \\
&\quad = \mathrm{E}\left[\mathrm{E}\left[(g(X_*; \mathcal{D}_{\text{train}}) - Y_*)^2 \mid \mathcal{D}_{\text{train}}\right]\right] && \text{(prediction rule)} \\
&\quad = \mathrm{E}\left[(g(X_*; \mathcal{D}_{\text{train}}) - Y_*)^2\right] && \text{(iterated expectations)} \\
&\quad = \mathrm{E}\left[\mathrm{E}\left[(g(X_*; \mathcal{D}_{\text{train}}) - Y_*)^2 \mid X_*\right]\right] && \text{(iterated expectations)} \\
&\quad = \mathrm{E}[\mathrm{EPE}_n^g(X_*)] && \text{(definition)}
\end{aligned}
$$

The steps labeled "definition" apply Equation 11, Equation 7 and Equation 13, in that order, and the step labeled "prediction rule" applies Equation 6, $\hat{y}(X_*) = g(X_*; \mathcal{D}_{\text{train}})$. The two steps labeled "iterated expectations" use the law of iterated expectations, the first conditioning on $\mathcal{D}_{\text{train}}$ and the second on X_*. The last line needs $\mathcal{D}_{\text{train}}$ to be independent of (X_*, Y_*): then, given $X_* = x$, $\mathcal{D}_{\text{train}}$ still has its unconditional distribution, as Definition 0.14 requires.

Theorem (The expected generalization error averages the error at each covariate value)

Theorem 0.1 (The expected generalization error averages the error at each covariate value)**.** *If the random training set $\mathcal{D}_{train}$ is independent of the new observation (X_*, Y_*), the expected generalization error of a fitting procedure g is the mean of its expected squared prediction error at x over the distribution of X_*:*

$$\mathrm{Err}^g(n) = \mathrm{E}[\mathrm{EPE}_n^g(X_*)]. \tag{14}$$

Proof

Proof. This is Solution 0.3. □

Definition (Bias of a prediction)

Definition 0.15 (Bias of a prediction)**.** Let g be a fitting procedure, and let $\mathcal{D}_{\text{train}}$ be a random training set. At a covariate value x, the prediction $g(x; \mathcal{D}_{\text{train}})$ is a random variable, because it depends on the random training data. The **bias** of the prediction, as a prediction of $\mu(x)$, is the bias of $g(x; \mathcal{D}_{\text{train}})$ as an estimator of the mean function value $\mu(x)$:

$$\text{Bias}(g(x; \mathcal{D}_{\text{train}})) = \text{E}[g(x; \mathcal{D}_{\text{train}})] - \mu(x). \tag{15}$$

Remark

Remark. In expected prediction error decompositions (James et al. 2021, sec. 2.2.2), the bias reflects the error introduced by approximating a real-world relationship by a model.

Example (Computing the bias of a prediction)

Example 0.3 (Computing the bias of a prediction)**.** Suppose $\mu(x) = 10$, and the predictions $g(x; \mathcal{D}_{\text{train}})$, over repeated training sets, average 11. Then

$$\begin{aligned}\text{Bias}(g(x; \mathcal{D}_{\text{train}})) &= 11 - 10 \\ &= 1.\end{aligned}$$

The squared bias is $1^2 = 1$, the value in Exercise 0.4.

Exercise (Expected squared error from three sources)

Exercise 0.4 (Expected squared error from three sources)**.** A prediction $g(x; \mathcal{D}_{\text{train}})$ at a covariate value x has squared bias $[\text{Bias}(g(x; \mathcal{D}_{\text{train}}))]^2 = 1$ and variance $\text{Var}(g(x; \mathcal{D}_{\text{train}})) = 2$. The new outcome is $Y_* = \mu(x) + \varepsilon$, where ε has mean 0 and variance $\sigma^2 = 3$ and is independent of $g(x; \mathcal{D}_{\text{train}})$.

1. What is the expected squared prediction error $\text{E}\big[(Y_* - g(x; \mathcal{D}_{\text{train}}))^2\big]$?
2. If we could remove all bias and all variance of $g(x; \mathcal{D}_{\text{train}})$, what would the expected squared prediction error be?
3. Which of the three quantities (squared bias, variance of $g(x; \mathcal{D}_{\text{train}})$, σ^2) can a better prediction method reduce, and which cannot?

Solution

Solution 0.4.

1. The expected squared error is the sum of the three sources: $1 + 2 + 3 = 6$.
2. $0 + 0 + 3 = 3$. The noise variance $\sigma^2 = 3$ remains, so 3 is the smallest expected squared error any prediction can have here.
3. A better method can reduce the squared bias and the variance of $g(x; \mathcal{D}_{\text{train}})$, because both depend on how g is built. It cannot reduce σ^2, which is the variance of the noise ε in the new outcome.

Theorem (Bias, variance and noise)

Theorem 0.2 (Bias, variance and noise)**.** *Let μ be the mean function of the outcome given the covariate. Suppose that, given $X_* = x$, the new outcome is $Y_* = \mu(x) + \varepsilon$, where ε, the noise in the outcome at x, has mean 0 and variance σ^2, and that Y_* is independent of the random training set $\mathcal{D}_{train}$, and that $\mathrm{E}[g(x;\mathcal{D}_{train})^2] < \infty$. Then the expected squared prediction error at x is the squared bias of the prediction $g(x;\mathcal{D}_{train})$ as a prediction of $\mu(x)$, plus its variance, plus σ^2:*

$$\mathrm{EPE}_n^g(x) = \left(\mathrm{Bias}(g(x;\mathcal{D}_{train}))\right)^2 + \mathrm{Var}(g(x;\mathcal{D}_{train})) + \sigma^2. \tag{16}$$

Proof

Proof. Write $\delta_f(x) \overset{\text{def}}{=} \mu(x) - g(x;\mathcal{D}_{\text{train}})$, the error of the prediction as an estimate of $\mu(x)$. Given $X_* = x$, the noise has mean 0 by Equation 12 and variance σ^2, and it is independent of $g(x;\mathcal{D}_{\text{train}})$ because Y_* is independent of $\mathcal{D}_{\text{train}}$; hence it is independent of $\delta_f(x)$, because $\mu(x)$ is a constant.
First, the prediction error in terms of ε and $\delta_f(x)$:

$$\begin{aligned} Y_* - g(x;\mathcal{D}_{\text{train}}) &= (\mu(x)+\varepsilon) - g(x;\mathcal{D}_{\text{train}}) && \text{(definition of } Y_*) \\ &= \varepsilon + (\mu(x) - g(x;\mathcal{D}_{\text{train}})) && \text{(reorder the terms of the sum)} \\ &= \varepsilon + \delta_f(x) && \text{(definition of } \delta_f(x)) \end{aligned}$$

Next, expand the square and split the expectation. The first line is the definition of the expected squared prediction error at x, which conditions on $X_* = x$; all expectations in the lines after it are conditional on $X_* = x$ too. The last two steps use linearity of expectation, first for the expectation of a sum and then for a constant factor:

$$\begin{aligned} \mathrm{EPE}_n^g(x) &= \mathrm{E}\Big[\big(\varepsilon + \delta_f(x)\big)^2 \mid X_* = x\Big] && \text{(definition and the error above)} \\ &= \mathrm{E}\Big[\varepsilon^2 + 2\varepsilon\delta_f(x) + \big(\delta_f(x)\big)^2\Big] && \text{(expand the square of a sum)} \\ &= \mathrm{E}[\varepsilon^2] + \mathrm{E}\big[2\varepsilon\delta_f(x)\big] + \mathrm{E}\Big[\big(\delta_f(x)\big)^2\Big] && \text{(expectation of a sum)} \\ &= \mathrm{E}[\varepsilon^2] + 2\,\mathrm{E}\big[\varepsilon\delta_f(x)\big] + \mathrm{E}\Big[\big(\delta_f(x)\big)^2\Big] && \text{(constant factor 2)} \end{aligned}$$

Each of the three terms is evaluated in turn.

- For the noise term, the variance identity gives $\sigma^2 = \mathrm{E}[\varepsilon^2] - (\mathrm{E}[\varepsilon])^2$, and $\mathrm{E}[\varepsilon] = 0$, so $\mathrm{E}[\varepsilon^2] = \sigma^2$.
- For the cross term, ε is independent of $\delta_f(x)$, so the expectation of the product is the product of the expectations:

$$\begin{aligned}
\mathrm{E}\big[\varepsilon\delta_f(x)\big] &= \mathrm{E}[\varepsilon]\,\mathrm{E}\big[\delta_f(x)\big] && \text{(independence)}\\
&= 0 \cdot \mathrm{E}\big[\delta_f(x)\big] && (\mathrm{E}[\varepsilon] = 0)\\
&= 0 && \text{(zero times a number)}
\end{aligned}$$

- For the last term, the variance identity gives $\mathrm{E}\Big[\big(\delta_f(x)\big)^2\Big] = \mathrm{Var}\big(\delta_f(x)\big) + \big(\mathrm{E}\big[\delta_f(x)\big]\big)^2$. The mean of $\delta_f(x)$ is

$$\begin{aligned}
\mathrm{E}\big[\delta_f(x)\big] &= \mathrm{E}[\mu(x)] - \mathrm{E}[g(x;\mathcal{D}_{\text{train}})] && \text{(expectation of a difference)}\\
&= \mu(x) - \mathrm{E}[g(x;\mathcal{D}_{\text{train}})] && (\mu(x) \text{ is a constant})\\
&= -(\mathrm{E}[g(x;\mathcal{D}_{\text{train}})] - \mu(x)) && \text{(factor out } -1)\\
&= -\operatorname{Bias}(g(x;\mathcal{D}_{\text{train}})) && \text{(definition of bias)}
\end{aligned}$$

  so $\big(\mathrm{E}\big[\delta_f(x)\big]\big)^2 = (\operatorname{Bias}(g(x;\mathcal{D}_{\text{train}})))^2$, because a number and its negative have the same square. The variance of $\delta_f(x)$ is

$$\begin{aligned}
\mathrm{Var}\big(\delta_f(x)\big) &= \mathrm{E}\Big[\big(\delta_f(x) - \mathrm{E}\big[\delta_f(x)\big]\big)^2\Big] && \text{(definition of variance)}\\
&= \mathrm{E}\Big[\big(-(g(x;\mathcal{D}_{\text{train}}) - \mathrm{E}[g(x;\mathcal{D}_{\text{train}})])\big)^2\Big] && \text{(substitute; the constant } \mu(x) \text{ cancels)}\\
&= \mathrm{E}\Big[\big(g(x;\mathcal{D}_{\text{train}}) - \mathrm{E}[g(x;\mathcal{D}_{\text{train}})]\big)^2\Big] && \text{(a number and its negative have the same square}\\
&= \mathrm{Var}(g(x;\mathcal{D}_{\text{train}})) && \text{(definition of variance)}
\end{aligned}$$

Substituting the three terms into the expansion gives

$$\begin{aligned}
\mathrm{EPE}_n^g(x) &= \sigma^2 + 2\cdot 0 + \Big(\mathrm{Var}(g(x;\mathcal{D}_{\text{train}})) + (\operatorname{Bias}(g(x;\mathcal{D}_{\text{train}})))^2\Big) && \text{(the three terms above)}\\
&= (\operatorname{Bias}(g(x;\mathcal{D}_{\text{train}})))^2 + \mathrm{Var}(g(x;\mathcal{D}_{\text{train}})) + \sigma^2 && \text{(drop } 2\cdot 0\text{, reorder the terms)}
\end{aligned}$$

□

ℹ Definition (Irreducible error)

Definition 0.16 (Irreducible error)**.** In Theorem 0.2, the term $\sigma^2 = \mathrm{Var}(\varepsilon)$ is the **irreducible error**. It does not depend on the fitting procedure g, so no choice of prediction can remove it. The theorem concerns the single covariate value x, so σ^2 may differ from one covariate value to another: it does not assume constant noise variance across covariate values. For example, in Exercise 0.4, $\sigma^2 = 3$, so every prediction there has expected squared error at least 3.

Remark

Remark. This expected prediction error decomposition into squared bias, variance, and irreducible error follows James et al. (2021, sec. 2.2.2).

Remark (Other names, and how the expected error splits)

Remark 0.2 (Other names, and how the expected error splits). Hastie et al. (2009, sec. 7.2) also call $\mathrm{Err}_{\mathcal{D}_{\text{train}}}$ the *test error*, and in sec. 7.12 the *conditional* test error. It is the risk of the prediction $\widehat{y}(X_*)$ under squared error loss, computed with the training set held fixed.
The argument n of the expected generalization error $\mathrm{Err}^g(n)$ of a fitting procedure g records that it depends on how many observations each training set has. By Theorem 0.1, $\mathrm{Err}^g(n)$ averages the expected squared prediction error at each covariate value, and Theorem 0.2 splits that error into a squared bias, a variance and a noise variance.

Definition (Flexibility)

Definition 0.17 (Flexibility)**.** Let g_1 and g_2 be fitting procedures that each choose, by least squares, a prediction rule from a set of candidate functions, $\mathcal{G}_1$ and $\mathcal{G}_2$ respectively. The procedure g_2 is **at least as flexible** as g_1 if every candidate of g_1 is also a candidate of g_2:

$$\mathcal{G}_1 \subseteq \mathcal{G}_2. \tag{17}$$

Source

James et al. (2021, sec. 2.1.3) use flexibility without a formal definition.

Remark (Polynomial degree orders flexibility)

Remark 0.3 (Polynomial degree orders flexibility). Least-squares polynomial fits are ordered by degree: a polynomial of degree d is also a polynomial of degree $d+1$ whose coefficient of x^{d+1} is zero, so the fit of degree $d+1$ is at least as flexible as the fit of degree d. The four fits of Exercise 0.1 are ordered this way. Definition 0.17 is narrower than the informal use of the word in James et al. (2021, sec. 2.1.3): it orders only procedures whose candidate sets are nested. Other procedures can be compared by other measures, such as the effective number of parameters (Hastie et al. 2009, sec. 7.6).

Definition (Bias–variance trade-off)

Definition 0.18 (Bias–variance trade-off)**.** Let $g_1, \dots, g_L$ be fitting procedures, each at least as flexible as the one before, and let $\mathcal{D}_{\text{train}}$ be a random training set. At a covariate value x, with μ the mean function, there is a **bias–variance trade-off** among them when, from each procedure to the next, the squared bias of the prediction, as a prediction of $\mu(x)$, does not rise and its variance does not fall:

$$\begin{aligned} \left(\text{Bias}(g_{l+1}(x; \mathcal{D}_{\text{train}}))\right)^2 &\leq \left(\text{Bias}(g_l(x; \mathcal{D}_{\text{train}}))\right)^2, \\ \text{Var}(g_{l+1}(x; \mathcal{D}_{\text{train}})) &\geq \text{Var}(g_l(x; \mathcal{D}_{\text{train}})), \qquad l = 1, \dots, L-1. \end{aligned} \tag{18}$$

Source

James et al. (2021, sec. 2.2.2).

Remark (Overfitting and the trade-off)

Remark 0.4 (Overfitting and the trade-off). By Theorem 0.2, $\text{EPE}_n^{g_l}(x)$ is the squared bias plus the variance plus σ^2. Under a bias–variance trade-off among $g_1, \dots, g_L$, moving from g_l to g_{l+1} lowers the expected squared prediction error at x when the squared bias falls by more than the variance rises, and raises it otherwise. The smallest of $\text{EPE}_n^{g_1}(x), \dots, \text{EPE}_n^{g_L}(x)$ is often at some g_l with $1 < l < L$ (James et al. 2021, sec. 2.2.2), but not always: if the mean function μ is a candidate of g_1, the squared bias of g_1 can be zero, and then no later g_l has a smaller squared bias and, under the trade-off, none has a smaller variance, so none has a smaller expected squared prediction error. The procedures past the smallest error have lowered their squared bias by less than they have raised their variance, and the rules they produce tend to overfit: they fit their training sets more closely than the procedure with the smallest error but predict new observations worse.

Remark (Coefficients per observation)

Remark 0.5 (Coefficients per observation). Adding coefficients to a least-squares fit makes it at least as flexible as before. With few observations per coefficient, the variance this adds can outweigh the squared bias it removes, which is the overfitting side of Remark 0.4. In Exercise 0.1, each added power of `wt`, like an added predictor, is one more coefficient fitted to the same 20 cars: the degree-4 fit has five coefficients for those 20 cars.

Exercise (A held-out set estimates the generalization error)

Exercise 0.5 (A held-out set estimates the generalization error)**.** A prediction rule $\hat{y}$ is fitted to a training set $\mathcal{D}_{\text{train}}$. A held-out set of m observations $(X_{*,1}, Y_{*,1}), \dots, (X_{*,m}, Y_{*,m})$

is drawn from the same population as the new observation (X_*, Y_*) of Definition 0.7, each independently of $\mathcal{D}_{\text{train}}$.
Show that, given $\mathcal{D}_{\text{train}}$, the expected mean squared error of the held-out predictions is $\text{Err}_{\mathcal{D}_{\text{train}}}$:

$$\text{E}\left[\frac{1}{m}\sum_{j=1}^{m}\left(\hat{y}(X_{*,j}) - Y_{*,j}\right)^2 \mid \mathcal{D}_{\text{train}}\right] = \text{Err}_{\mathcal{D}_{\text{train}}}.$$

Solution

Solution 0.5. Given $\mathcal{D}_{\text{train}}$, the rule $\hat{y}$ is fixed. Each held-out pair has the same distribution as (X_*, Y_*) and is independent of $\mathcal{D}_{\text{train}}$, so each squared prediction error has conditional expectation $\text{E}\left[\left(\hat{y}(X_{*,j}) - Y_{*,j}\right)^2 \mid \mathcal{D}_{\text{train}}\right] = \text{Err}_{\mathcal{D}_{\text{train}}}$ by Equation 7. Then

$$\begin{aligned}
&\text{E}\left[\frac{1}{m}\sum_{j=1}^{m}\left(\hat{y}(X_{*,j}) - Y_{*,j}\right)^2 \mid \mathcal{D}_{\text{train}}\right] \\
&= \frac{1}{m}\,\text{E}\left[\sum_{j=1}^{m}\left(\hat{y}(X_{*,j}) - Y_{*,j}\right)^2 \mid \mathcal{D}_{\text{train}}\right] && \text{(constant } 1/m \text{ out)} \\
&= \frac{1}{m}\sum_{j=1}^{m}\text{E}\left[\left(\hat{y}(X_{*,j}) - Y_{*,j}\right)^2 \mid \mathcal{D}_{\text{train}}\right] && \text{(linearity of E)} \\
&= \frac{1}{m}\sum_{j=1}^{m}\text{Err}_{\mathcal{D}_{\text{train}}} && \text{(each term, as above)} \\
&= \frac{1}{m}\left(m\ \text{Err}_{\mathcal{D}_{\text{train}}}\right) && (m \text{ equal terms}) \\
&= \left(\frac{1}{m}\cdot m\right)\text{Err}_{\mathcal{D}_{\text{train}}} && \text{(regroup the product)} \\
&= 1\cdot\text{Err}_{\mathcal{D}_{\text{train}}} && (\tfrac{1}{m}\cdot m = 1) \\
&= \text{Err}_{\mathcal{D}_{\text{train}}} && \text{(multiplying by 1 changes nothing)}
\end{aligned}$$

The step labeled "each term, as above" is the one that needs the held-out set to be independent of $\mathcal{D}_{\text{train}}$.

Theorem (Held-out mean squared error estimates the generalization error)

Theorem 0.3 (Held-out mean squared error estimates the generalization error)**.** *Let a prediction rule $\hat{y}$ be fitted to a training set $\mathcal{D}_{train}$, and let a held-out set of m observations $(X_{*,1}, Y_{*,1}), \ldots, (X_{*,m}, Y_{*,m})$ be drawn from the same population as (X_*, Y_*), each independently of $\mathcal{D}_{train}$. Then, given $\mathcal{D}_{train}$, the expected mean squared error of the held-out*

predictions is the generalization error:

$$\mathrm{E}\left[\frac{1}{m}\sum_{j=1}^{m}\left(\hat{y}(X_{*,j}) - Y_{*,j}\right)^2 \mid \mathcal{D}_{train}\right] = \mathrm{Err}_{\mathcal{D}_{train}}.$$

Proof

Proof. This is Solution 0.5. □

Remark (Training error is optimistic)

Remark 0.6 (Training error is optimistic). The same argument fails for the training set. The training pairs are not independent of $\mathcal{D}_{\text{train}}$: they are $\mathcal{D}_{\text{train}}$, and $\hat{y}$ was fitted to make their prediction errors small. So the step of Solution 0.5 labeled "each term, as above" does not hold for them, and the training error $\overline{\text{err}}$ is typically *smaller* than $\mathrm{Err}_{\mathcal{D}_{\text{train}}}$ (Hastie et al. 2009, sec. 7.4). Exercise 0.1 shows the pattern: training RMSE keeps falling as the degree grows, while held-out RMSE does not.

Example (Training error and generalization error by degree)

Example 0.4 (Training error and generalization error by degree)**.** Real data never reveal $\mathrm{Err}_{\mathcal{D}_{\text{train}}}$, but a simulation can, because it can draw as many new observations as we like. Here the population is known: $X \sim \text{Uniform}(0,1)$ and $Y \mid X = x \sim \mathrm{N}(\sin(2\pi x), \sigma^2)$, with $\sigma = 0.3$. We draw one training set of $n = 30$ observations, fit a polynomial in x of each degree from 1 to 10 by least squares, and score each fit on the training set and on 10,000 new observations. By Theorem 0.3, the second score has expectation $\mathrm{Err}_{\mathcal{D}_{\text{train}}}$ given the training set, and with 10,000 observations it is a precise estimate of it.

```r
sim_sigma <- 0.3
sim_truth <- function(x) sin(2 * pi * x)
sim_draw <- function(n) {
  x <- stats::runif(n)
  tibble::tibble(x = x, y = sim_truth(x) + stats::rnorm(n, sd = sim_sigma))
}

set.seed(1)
sim_train <- sim_draw(30)
sim_new <- sim_draw(10000)
sim_degrees <- 1:10

sim_errors <- tibble::tibble(degree = sim_degrees) |>
  dplyr::mutate(
    fit = purrr::map(degree, \(d) lm(y ~ poly(x, d), data = sim_train)),
    training = purrr::map_dbl(fit, rmse, data = sim_train),
    generalization = purrr::map_dbl(fit, rmse, data = sim_new)
  ) |>
  dplyr::select(-fit)

sim_errors
#> # A tibble: 10 x 3
#>    degree training generalization
#>     <int>    <dbl>          <dbl>
#>  1      1    0.567          0.547
#>  2      2    0.560          0.557
#>  3      3    0.231          0.315
#>  4      4    0.231          0.315
#>  5      5    0.216          0.306
#>  6      6    0.195          0.324
#>  7      7    0.194          0.326
#>  8      8    0.184          0.335
#>  9      9    0.183          0.335
#> 10     10    0.180          0.341
```

```
sim_errors |>
  tidyr::pivot_longer(
    c(training, generalization),
    names_to = "error",
    values_to = "rmse"
  ) |>
  ggplot2::ggplot(ggplot2::aes(degree, rmse, colour = error, shape = error)) +
  ggplot2::geom_hline(yintercept = sim_sigma, linetype = "dashed") +
  ggplot2::geom_line() +
  ggplot2::geom_point(size = 2) +
  ggplot2::scale_x_continuous(breaks = sim_degrees) +
  ggplot2::scale_y_log10() +
  ggplot2::labs(
    x = "polynomial degree",
    y = "root mean squared error (log scale)",
    colour = NULL,
    shape = NULL
  ) +
  ggplot2::theme_bw() +
  ggplot2::theme(legend.position = "top")
```

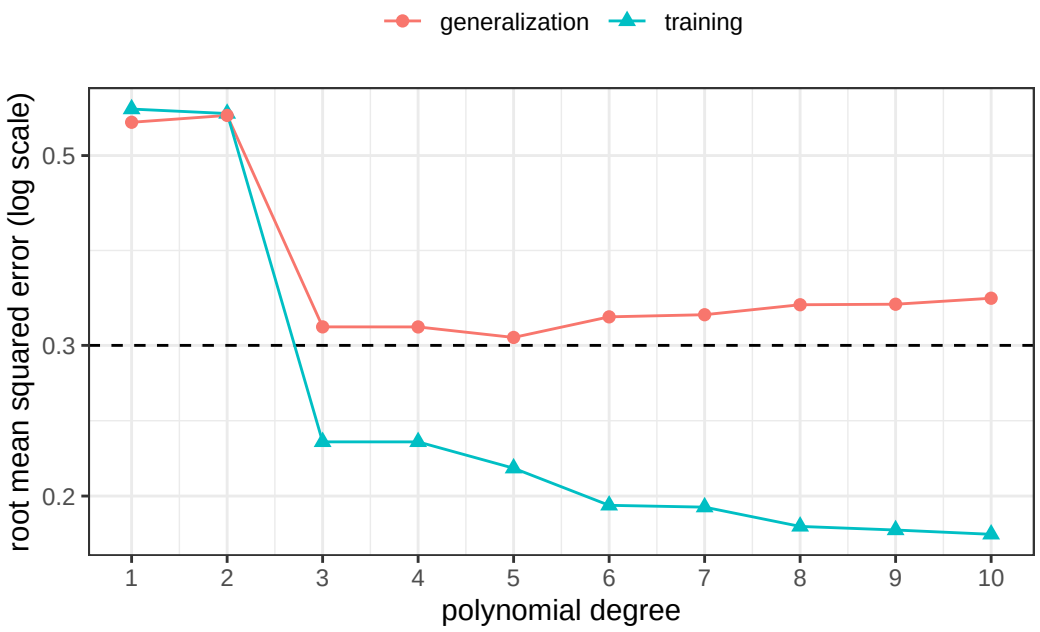


Figure 1: Training RMSE and estimated generalization error (RMSE on 10,000 new observations) of polynomial fits to one simulated training set of $n = 30$. The dashed line is the noise standard deviation σ, the square root of the irreducible error.

```r
sim_best_generalization <-
  sim_errors$degree[which.min(sim_errors$generalization)]
sim_best_training <- sim_errors$degree[which.min(sim_errors$training)]
sim_n_below <- c(
  training = sum(sim_errors$training < sim_sigma),
  generalization = sum(sim_errors$generalization < sim_sigma)
)
c(
  best_by_training = sim_best_training,
  best_by_generalization = sim_best_generalization,
  n_below_sigma2 = sim_n_below
)
#>              best_by_training        best_by_generalization
#>                            10                             5
#>       n_below_sigma2.training n_below_sigma2.generalization
#>                             8                             0
```

The training error is smallest at degree 10 and never rises as the degree grows, because each polynomial family contains the one before it. The estimated generalization error is smallest at degree 5. No prediction rule can have generalization RMSE below $\sigma = 0.3$. Yet the training RMSE falls below σ at 8 of the ten degrees, while the estimated generalization error falls below it at 0 of them: the training error is optimistic.

Training, validation, and test sets

Exercise (Sizes of a three-way split)

Exercise 0.6 (Sizes of a three-way split)**.** The `mtcars` data frame has $n = 32$ cars. Suppose we assign $\lfloor 0.6n \rfloor$ cars to a training set, $\lfloor 0.2n \rfloor$ cars to a validation set, and all remaining cars to a test set. How many cars are in each set?

Solution

Solution 0.6.

$$\begin{aligned} \lfloor 0.6 \cdot 32 \rfloor &= \lfloor 19.2 \rfloor \\ &= 19 \end{aligned}$$

training cars,

$$\begin{aligned}\lfloor 0.2 \cdot 32 \rfloor &= \lfloor 6.4 \rfloor \\ &= 6\end{aligned}$$

validation cars, and $32 - 19 - 6 = 7$ test cars.

Definition (Training/validation/test split)

Definition 0.19 (Training/validation/test split)**.** A **training/validation/test split** divides the observations $1, \dots, n$ of a data set into three disjoint sets of indices, I_{train}, I_{val} and I_{test}, and takes the observations in I_{train} as the training set:

$$\begin{aligned} &I_{\text{train}} \cup I_{\text{val}} \cup I_{\text{test}} = \{1, \dots, n\}, \\ &I_{\text{train}} \cap I_{\text{val}} = \emptyset, \\ &I_{\text{train}} \cap I_{\text{test}} = \emptyset, \\ &I_{\text{val}} \cap I_{\text{test}} = \emptyset, \\ &\mathcal{D}_{\text{train}} = ((x_i, y_i))_{i \in I_{\text{train}}}. \end{aligned} \tag{19}$$

Source

James et al. (2021, 198–201).

Example (Split sizes for 32 cars)

Example 0.5 (Split sizes for 32 cars)**.** In Exercise 0.6, a rule of 60% / 20% / 20% on $n = 32$ cars gives sets of 19, 6, and 7 cars, and the 7 test cars are used only once, after the model is chosen.

Definition (Tuning parameter)

Definition 0.20 (Tuning parameter)**.** A **tuning parameter** of a fitting procedure is a setting λ that the procedure needs but does not compute from the training set:

$$\hat{y}(x) = g_\lambda(x; \mathcal{D}_{\text{train}}). \tag{20}$$

Example (Polynomial degree)

Example 0.6 (Polynomial degree)**.** In Exercise 0.1, least squares computes the coefficients of each polynomial from the 20 training cars, but the degree d is fixed before the fit. So the degree is a tuning parameter of least-squares polynomial fitting, and the four fits are g_1, g_2, g_3, g_4 with $\lambda = d$.

Definition (Validation set)

Definition 0.21 (Validation set)**.** The **validation set** of a training/validation/test split is the held-out set of observations indexed by I_{val}, used to choose among candidate fitting procedures fitted to the training set:

$$((x_i, y_i))_{i \in I_{\text{val}}}. \tag{21}$$

Definition (Validation choice)

Definition 0.22 (Validation choice)**.** Given candidate fitting procedures $g_1, \dots, g_L$, the **validation choice** $\hat{l}$ is the index of the candidate whose prediction rule, fitted to the training set $\mathcal{D}_{\text{train}}$, has the smallest mean squared error on the validation set:

$$\hat{l} \stackrel{\text{def}}{=} \arg\min_{l} \frac{1}{|I_{\text{val}}|} \sum_{i \in I_{\text{val}}} (g_l(x_i; \mathcal{D}_{\text{train}}) - y_i)^2. \tag{22}$$

Definition (Test set)

Definition 0.23 (Test set)**.** The **test set** of a training/validation/test split is the held-out set of observations indexed by I_{test}, on which the rule of the validation choice $g_{\hat{l}}$ is scored:

$$((x_i, y_i))_{i \in I_{\text{test}}}. \tag{23}$$

Remark (What each set is for)

Remark 0.7 (What each set is for). Each set of a training/validation/test split has one job:

- the training set is used to fit each candidate procedure, which estimates its parameters;
- the validation set is used to compare the candidates, choose transformations, or choose values of a tuning parameter;
- the test set is used once, after the validation choice is made, to estimate the generalization error of the chosen rule.

The chosen procedure won on the validation set partly because of that particular validation sample, so its validation error is optimistic. Keeping the test set out of every choice is what makes its score free of that optimism.
For further reading, the machine learning notes define the same three sets in their own notation: the training set, the validation set, and the test set. They use a different symbol

for the training set and a parenthesized observation index, but the same three-way split.

Example (Numerical example)

Example 0.7 (Numerical example)**.** This example (Example 0.7) uses R's built-in `mtcars` dataset ($n = 32$ cars) to predict fuel efficiency (`mpg`) from vehicle weight (`wt`). It uses one random split to compare linear, quadratic, cubic, and quartic models (`mpg ~ wt`, `mpg ~ wt + I(wt^2)`, `mpg ~ wt + I(wt^2) + I(wt^3)`, and `mpg ~ wt + I(wt^2) + I(wt^3) + I(wt^4)`) on a validation set, then reports the chosen model's test RMSE on untouched test data (James et al. 2021, 213).

Table 1: Sizes of the training, validation, and test splits

```
set.seed(108)
n <- nrow(mtcars)

n_train <- floor(0.6 * n)
n_valid <- floor(0.2 * n)

idx_train <- sample.int(n, size = n_train)
idx_remaining <- setdiff(seq_len(n), idx_train)
idx_valid <- sample(idx_remaining, size = n_valid)
idx_test <- setdiff(idx_remaining, idx_valid)

split_sizes <- tibble::tibble(
  split = c("training", "validation", "test"),
  n = c(length(idx_train), length(idx_valid), length(idx_test))
)

split_sizes
#> # A tibble: 3 x 2
#>   split          n
#>   <chr>      <int>
#> 1 training      19
#> 2 validation     6
#> 3 test           7
```

```
train_dat <- mtcars[idx_train, ]
valid_dat <- mtcars[idx_valid, ]
test_dat <- mtcars[idx_test, ]

model_linear <- lm(mpg ~ wt, data = train_dat)
model_quadratic <- lm(mpg ~ wt + I(wt^2), data = train_dat)
model_cubic <- lm(mpg ~ wt + I(wt^2) + I(wt^3), data = train_dat)
model_quartic <- lm(
  mpg ~ wt + I(wt^2) + I(wt^3) + I(wt^4),
  data = train_dat
)

candidate_models <- list(
  linear = model_linear,
  quadratic = model_quadratic,
  cubic = model_cubic,
  quartic = model_quartic
)

validation_results <- tibble::tibble(
  model = names(candidate_models)
) |>
  dplyr::mutate(
    training_RMSE = purrr::map_dbl(
      model,
      \(model_name) rmse(candidate_models[[model_name]], train_dat)
    ),
    validation_RMSE = purrr::map_dbl(
      model,
      \(model_name) rmse(candidate_models[[model_name]], valid_dat)
    )
  )

model_labels <- c(
  linear = "linear model",
  quadratic = "quadratic model",
  cubic = "cubic model",
  quartic = "quartic model"
)

chosen_model_name <-
  validation_results |>
  dplyr::arrange(validation_RMSE) |>
  dplyr::slice(1) |>
  dplyr::pull(model)
```



```
chosen_model_label <- model_labels[[chosen_model_name]]

chosen_model <- candidate_models[[chosen_model_name]]
chosen_model_validation_rmse <- rmse(chosen_model, valid_dat)

performance_comparison <- tibble::tibble(
```

```r
partition_colors <- c(
  training = "#1b9e77",
  validation = "#d95f02",
  test = "#7570b3"
)
partition_shapes <- c(
  training = 16,
  validation = 17,
  test = 15
)
wt_range <- range(c(train_dat$wt, valid_dat$wt, test_dat$wt))
mpg_range <- range(c(train_dat$mpg, valid_dat$mpg, test_dat$mpg))
wt_grid <- seq(wt_range[1], wt_range[2], length.out = 200)

partition_data <- dplyr::bind_rows(
  training = train_dat,
  validation = valid_dat,
  test = test_dat,
  .id = "series"
)

plot_partitions_with_fit <- function(model, fit_label) {
  series_levels <- c(names(partition_colors), fit_label)
  partition_data <- partition_data |>
    dplyr::mutate(
      series = factor(partition_data$series, levels = series_levels)
    )
  grid_data <- tibble::tibble(wt = wt_grid)
  fitted_curve <- tibble::tibble(
    wt = wt_grid,
    mpg = predict(model, newdata = grid_data),
    series = factor(fit_label, levels = series_levels)
  )
  # nolint start: object_usage_linter.
  ggplot2::ggplot(
    partition_data,
    ggplot2::aes(
      x = wt,
      y = mpg,
      color = series,
      shape = series
    )
  ) +
    ggplot2::geom_point() +
    ggplot2::geom_line(
      ggplot2::aes(linetype = series),
      data = fitted_curve,
      linewidth = 1
    ) +
    ggplot2::scale_color_manual(
      limits = series_levels,
      values = c(partition_colors, stats::setNames("blue", fit_label))
```

```
plot_partitions_with_fit(model_linear, "linear fit")
```

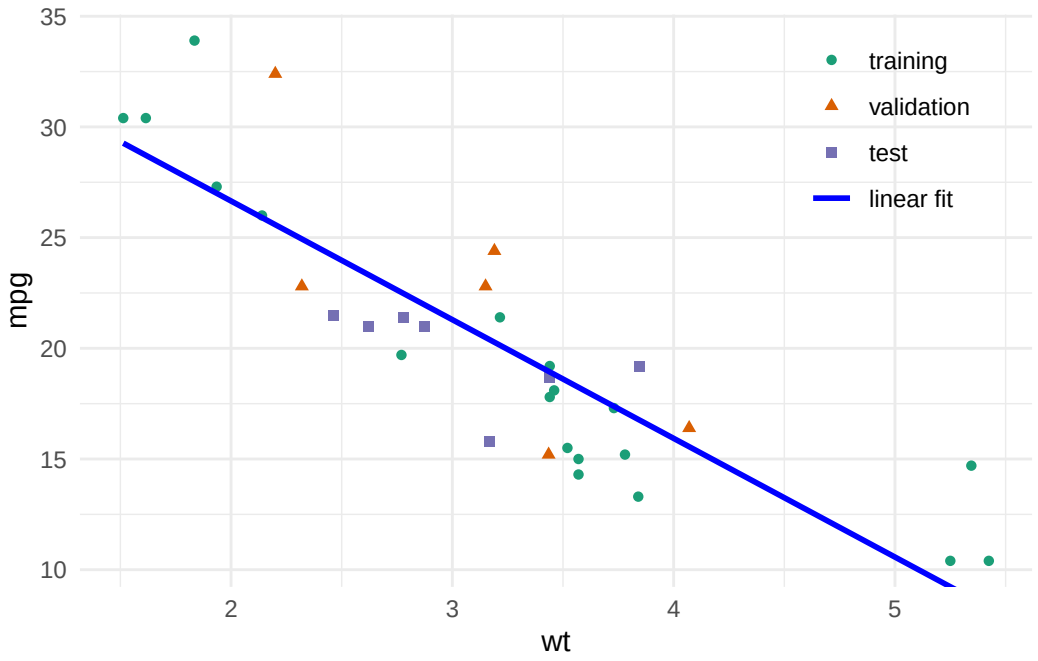


Figure 2: Linear model fit superimposed on data partitions

```
plot_partitions_with_fit(model_quadratic, "quadratic fit")
```

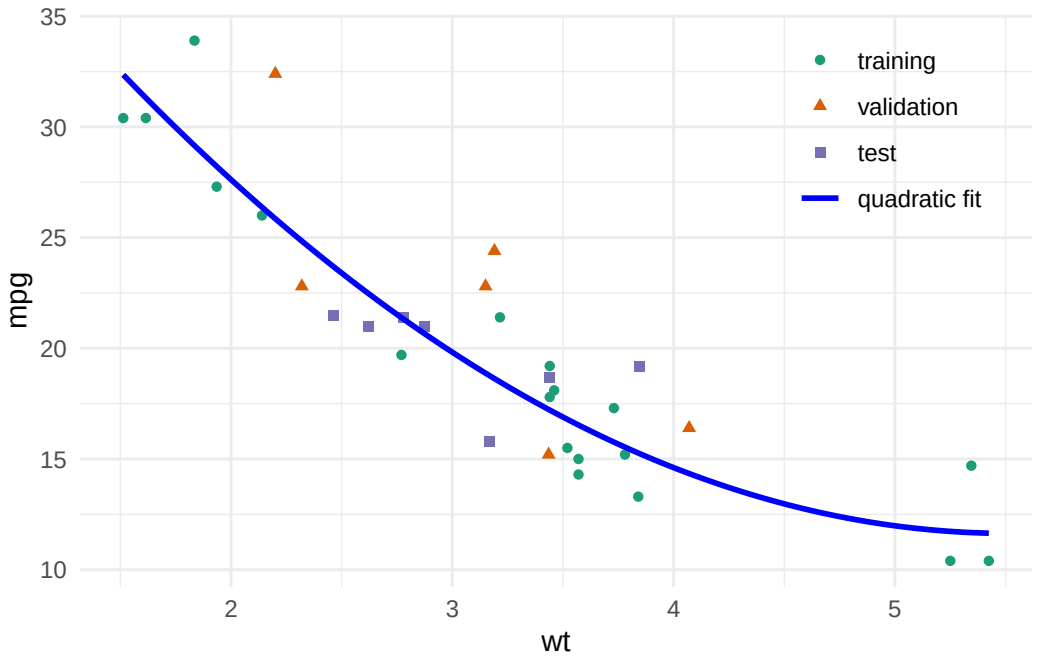


Figure 3: Quadratic model fit superimposed on data partitions

```
plot_partitions_with_fit(model_cubic, "cubic fit")
```

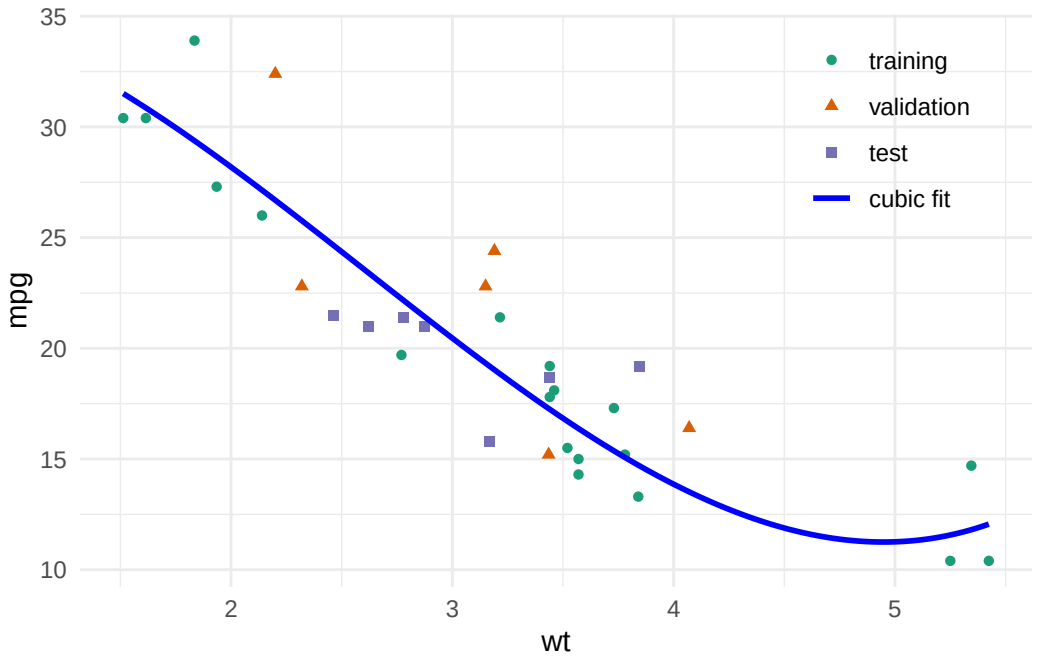


Figure 4: Cubic model fit superimposed on data partitions

```
plot_partitions_with_fit(model_quartic, "quartic fit")
```

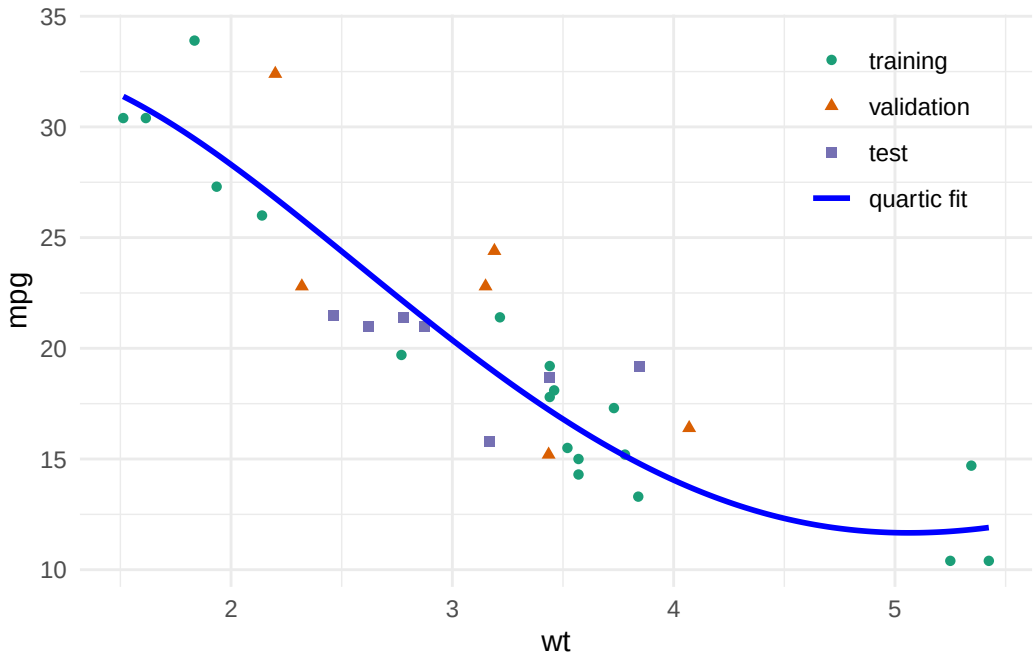


Figure 5: Quartic model fit superimposed on data partitions

Table 2: Training and validation RMSE for candidate models

```
validation_results
#> # A tibble: 4 x 3
#>   model     training_RMSE validation_RMSE
#>   <chr>             <dbl>           <dbl>
#> 1 linear             2.80            3.84
#> 2 quadratic          1.96            4.16
#> 3 cubic              1.91            3.98
#> 4 quartic            1.90            3.97
```

The chosen model is the linear model, because it has the lowest validation RMSE of the four. This split uses different cars from Exercise 0.1, and its validation set has only 6 cars, so the model it picks can differ from the one chosen there.

Table 3: Validation and test RMSE for the chosen model

```
performance_comparison
#> # A tibble: 2 x 3
#>   split      model   RMSE
#>   <chr>      <chr>  <dbl>
#> 1 validation linear  3.84
#> 2 test       linear  2.44
```

Source

The validation-set lab of James et al. (2021, 213).

k-fold cross-validation

Exercise (Folds of the `mtcars` data)

Exercise 0.7 (Folds of the `mtcars` data)**.** Suppose we use $k = 4$-fold cross-validation on the $n = 32$ cars in `mtcars`.

1. How many cars are in each fold?
2. Each time a model is fit, how many cars are used to fit it?
3. How many models are fit in total?
4. How many times is a given car used for fitting, and how many times for prediction?

Solution

Solution 0.7*.*

1. $32/4 = 8$ cars per fold.
2. Each fit leaves out one fold, so it uses $32 - 8 = 24$ cars.
3. One model per fold, so 4 models.
4. A given car is in exactly one fold. It is used for prediction once, when its fold is left out, and for fitting in the other 3 fits.

Definition (Folds)

Definition 0.24 (Folds)**.** A division of a training set $\mathcal{D}_{\text{train}}$, indexed by I, into k **folds** is a partition of I into k nonempty, disjoint subsets $\mathcal{F}_1, \dots, \mathcal{F}_k$:

$$\mathcal{F}_1 \cup \cdots \cup \mathcal{F}_k = I. \tag{24}$$

Source

James et al. (2021, sec. 5.1.3) and Hastie et al. (2009, sec. 7.10.1), who split the data into k roughly equal-sized parts.

Definition (Fold size)

Definition 0.25 (Fold size)**.** The **size** n_j of fold $\mathcal{F}_j$ is the number of observations it contains:

$$n_j \stackrel{\text{def}}{=} |\mathcal{F}_j|. \tag{25}$$

Definition (Fold assignment)

Definition 0.26 (Fold assignment)**.** Given folds $\mathcal{F}_1, \dots, \mathcal{F}_k$, the **fold assignment** $\kappa(i)$ of observation i is the index of the fold that contains it:

$$i \in \mathcal{F}_{\kappa(i)}. \tag{26}$$

Source

The fold-assignment function $\kappa(i)$ follows Hastie et al. (2009, sec. 7.10.1).

Definition (Fold-out rule)

Definition 0.27 (Fold-out rule)**.** For fold $\mathcal{F}_j$, the **fold-out rule** $\hat{y}^{(-j)}(\cdot)$ is the prediction rule that a fitting procedure g produces from the observations outside that fold:

$$\hat{y}^{(-j)}(x) \stackrel{\text{def}}{=} g\Big(x; ((x_i, y_i))_{i \in I \setminus \mathcal{F}_j}\Big). \tag{27}$$

Source

Hastie et al. (2009, sec. 7.10.1), who write the fitted function with the kth part of the data removed as $\hat{f}^{-k}$.

Definition (k-fold cross-validation)

Definition 0.28 (k-fold cross-validation)**.** In **k-fold cross-validation**:

1. The data are randomly divided into k folds of equal, or nearly equal, size.
2. For each fold $\mathcal{F}_j$, $j = 1, \dots, k$:
   - Fit the model to all data *except* fold $\mathcal{F}_j$, giving the fold-out rule $\widehat{y}^{(-j)}(\cdot)$.
   - Compute predicted values for the observations in fold $\mathcal{F}_j$.
3. Compute a summary prediction-error measure across all folds.

Remark (Cross-validation instead of a single validation set)

Remark 0.8 (Cross-validation instead of a single validation set). If data are limited, k-fold cross-validation can replace the single validation set of Definition 0.19 for more stable tuning. In Exercise 0.7, $k = 4$ folds of 8 cars each give 4 fits on 24 cars each.

Definition (Cross-validation estimate)

Definition 0.29 (Cross-validation estimate)**.** The **k-fold cross-validation estimate** $\widehat{\mathrm{Err}}_{\mathrm{CV}(k)}$ is the mean squared error of the predictions that each observation $i \in I$ of the training set receives from the fold-out rule of its fold assignment $\kappa(i)$ in k-fold cross-validation:

$$\widehat{\mathrm{Err}}_{\mathrm{CV}(k)} \stackrel{\mathrm{def}}{=} \frac{1}{n} \sum_{i \in I} \left(\widehat{y}^{(-\kappa(i))}(x_i) - y_i \right)^2. \tag{28}$$

Source

Hastie et al. (2009, sec. 7.10.1), for the per-observation form with the fold index $\kappa(i)$.

Exercise (The cross-validation estimate, fold by fold)

Exercise 0.8 (The cross-validation estimate, fold by fold)**.** Rewrite the cross-validation estimate Equation 28 as $1/n$ times a double sum: an outer sum over the fold numbers $j = 1, \dots, k$ and an inner sum over the observations i in fold $\mathcal{F}_j$. Then simplify the summand, so that it no longer uses the fold assignment $\kappa(i)$.

Solution

Solution 0.8. The folds $\mathcal{F}_1, \ldots, \mathcal{F}_k$ partition I (Definition 0.24), so summing over the observations of each fold and then over the folds adds each term of Equation 28 exactly once. For $i \in \mathcal{F}_j$, Equation 26 also puts i in $\mathcal{F}_{\kappa(i)}$, and the folds are disjoint, so $\kappa(i) = j$. Then

$$\begin{aligned}
\widehat{\mathrm{Err}}_{\mathrm{CV}(k)} &= \frac{1}{n}\sum_{i \in I}\left(\hat{y}^{(-\kappa(i))}(x_i) - y_i\right)^2 && \text{(definition)} \\
&= \frac{1}{n}\sum_{j=1}^{k}\sum_{i \in \mathcal{F}_j}\left(\hat{y}^{(-\kappa(i))}(x_i) - y_i\right)^2 && \text{(group by fold)} \\
&= \frac{1}{n}\sum_{j=1}^{k}\sum_{i \in \mathcal{F}_j}\left(\hat{y}^{(-j)}(x_i) - y_i\right)^2 && (\kappa(i) = j \text{ for } i \in \mathcal{F}_j)
\end{aligned}$$

In the last line, the rule $\hat{y}^{(-j)}(\cdot)$ depends only on the outer index, so each inner sum uses a single fitted rule.

Theorem (The cross-validation estimate, fold by fold)

Theorem 0.4 (The cross-validation estimate, fold by fold)**.** *The cross-validation estimate sums, fold by fold, the squared errors of each fold's observations under the rule fitted without that fold, and divides the total by n:*

$$\widehat{\mathrm{Err}}_{CV(k)} = \frac{1}{n}\sum_{j=1}^{k}\sum_{i \in \mathcal{F}_j}\left(\hat{y}^{(-j)}(x_i) - y_i\right)^2. \tag{29}$$

Proof

Proof. This is Solution 0.8. □

Definition (Fold mean squared error)

Definition 0.30 (Fold mean squared error)**.** The **fold mean squared error** MSE_j of fold $\mathcal{F}_j$ is the mean squared error of the fold-out rule's predictions for the n_j (fold size) observations in that fold:

$$\mathrm{MSE}_j \overset{\text{def}}{=} \frac{1}{n_j}\sum_{i \in \mathcal{F}_j}\left(\hat{y}^{(-j)}(x_i) - y_i\right)^2. \tag{30}$$

Source

James et al. (2021, sec. 5.1.3), who write the fold mean squared errors as $\mathrm{MSE}_1, \ldots, \mathrm{MSE}_k$.

ℹ Remark (What the cross-validation estimate estimates)

Remark 0.9 (What the cross-validation estimate estimates). Each prediction in Equation 28 comes from a rule that did not see the observation it predicts, so each squared error is scored as in Theorem 0.3. But the rule for fold $\mathcal{F}_j$ was fitted to $n - n_j$ observations, not n, and the k rules differ from one another and from the rule fitted to all n. So $\widehat{\mathrm{Err}}_{\mathrm{CV}(k)}$ estimates the expected generalization error of the fitting *procedure* g at the smaller training-set sizes $n - n_j$, which is $\mathrm{Err}^g(n(k-1)/k)$ when every fold has n/k observations, rather than $\mathrm{Err}_{\mathcal{D}_{\mathrm{train}}}$ for the one rule fitted to all the data (Hastie et al. 2009, sec. 7.12).
When every fold has $n_j = n/k$ observations, $\widehat{\mathrm{Err}}_{\mathrm{CV}(k)}$ is also the average of the k fold mean squared errors $\mathrm{MSE}_1, \ldots, \mathrm{MSE}_k$, the form James et al. (2021, sec. 5.1.3) use.

ℹ Remark (Choosing the number of folds)

Remark 0.10 (Choosing the number of folds). The number of folds k trades bias against variance in the cross-validation estimate itself (James et al. 2021, sec. 5.1.4):

- **Bias.** Each rule in Equation 28 is fitted to about $n(k-1)/k$ observations. A rule fitted to fewer observations usually predicts worse, so $\widehat{\mathrm{Err}}_{\mathrm{CV}(k)}$ tends to overstate the error of a rule fitted to all n. The overstatement shrinks as k grows.
- **Variance.** As k grows, the k training sets overlap more, so the k fitted rules are more alike and their errors are more strongly correlated. An average of strongly correlated errors varies more from one data set to another than an average of weakly correlated ones.
- **Computation.** The model is fitted k times.

$k = 5$ or $k = 10$ is the usual compromise.

ℹ Example (Cross-validation versus the generalization error)

Example 0.8 (Cross-validation versus the generalization error)**.** This example applies k-fold cross-validation with $k = 5$ to the simulated training set of Example 0.4, using only those 30 observations. The function `assign_folds()` assigns each observation to a fold at random, `cv_squared_errors()` fits the model once per fold without that fold and returns each observation's squared prediction error, and `cv_mse()` averages them, which is Equation 28.

```r
assign_folds <- function(n, k) {
  sample(rep_len(seq_len(k), n))
}

cv_squared_errors <- function(formula, data, folds) {
  y <- stats::model.response(stats::model.frame(formula, data))
  sq_errors <- numeric(nrow(data))
  for (j in unique(folds)) {
    in_fold <- folds == j
    fit <- lm(formula, data = data[!in_fold, ])
    pred <- predict(fit, newdata = data[in_fold, ])
    sq_errors[in_fold] <- (pred - y[in_fold])^2
  }
  sq_errors
}

cv_mse <- function(formula, data, folds) {
  mean(cv_squared_errors(formula, data, folds))
}

set.seed(5)
sim_folds <- assign_folds(nrow(sim_train), k = 5)
table(fold = sim_folds)
#> fold
#> 1 2 3 4 5
#> 6 6 6 6 6
```

The same fold assignment is used for every degree, so the degrees are compared on the same splits. The table and figure report the square root of each cross-validation estimate, so that it is in the same unit as the training and generalization RMSE of Example 0.4.

```
sim_errors_cv <- sim_errors |>
  dplyr::mutate(
    cv_5 = purrr::map_dbl(
      degree,
      \(d) sqrt(cv_mse(y ~ poly(x, d), sim_train, sim_folds))
    )
  )

sim_errors_cv
#> # A tibble: 10 x 4
#>    degree training generalization  cv_5
#>     <int>    <dbl>          <dbl> <dbl>
#>  1      1    0.567          0.547 0.597
#>  2      2    0.560          0.557 0.610
#>  3      3    0.231          0.315 0.287
#>  4      4    0.231          0.315 0.313
#>  5      5    0.216          0.306 0.323
#>  6      6    0.195          0.324 0.307
#>  7      7    0.194          0.326 0.395
#>  8      8    0.184          0.335 0.316
#>  9      9    0.183          0.335 0.536
#> 10     10    0.180          0.341 1.39
```

```
sim_errors_cv |>
  tidyr::pivot_longer(
    c(training, generalization, cv_5),
    names_to = "error",
    values_to = "rmse"
  ) |>
  dplyr::mutate(
    error = factor(
      error,
      levels = c("training", "cv_5", "generalization"),
      labels = c("training", "5-fold CV", "generalization")
    )
  ) |>
  ggplot2::ggplot(ggplot2::aes(degree, rmse, colour = error, shape = error)) +
  ggplot2::geom_hline(yintercept = sim_sigma, linetype = "dashed") +
  ggplot2::geom_line() +
  ggplot2::geom_point(size = 2) +
  ggplot2::scale_x_continuous(breaks = sim_degrees) +
  ggplot2::scale_y_log10() +
  ggplot2::labs(
    x = "polynomial degree",
    y = "root mean squared error (log scale)",
    colour = NULL,
    shape = NULL
  ) +
  ggplot2::theme_bw() +
  ggplot2::theme(legend.position = "top")
```

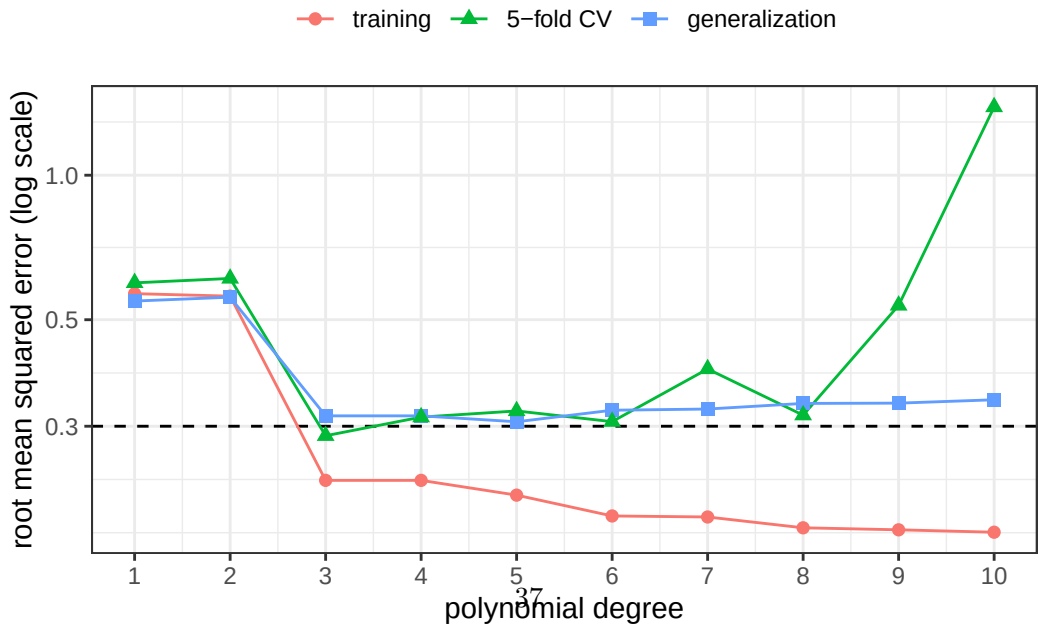


Figure 6: Figure 1 with the 5-fold cross-validation estimate added. Only the training and cross-validation curves could be computed from real data.

```r
sim_best_cv <- sim_errors_cv$degree[which.min(sim_errors_cv$cv_5)]
sim_n_fit <- nrow(sim_train) - max(table(sim_folds))
sim_top <- dplyr::filter(sim_errors_cv, degree == max(sim_degrees))
sim_cv_ratio_top <- sim_top$cv_5 / sim_top$generalization
c(
  best_by_cv = sim_best_cv,
  best_by_generalization = sim_best_generalization,
  n_per_fit = sim_n_fit,
  cv_over_generalization_at_top_degree = sim_cv_ratio_top
)
#>                            best_by_cv              best_by_generalization
#>                               3.00000                             5.00000
#>                             n_per_fit cv_over_generalization_at_top_degree
#>                              24.00000                             4.07316
```

The cross-validation estimate is smallest at degree 3, and the estimated generalization error at degree 5. Unlike the training error, the cross-validation estimate does not keep falling as the degree grows, so it does not simply reward higher degrees. It is not an estimate of $\mathrm{Err}_{\mathcal{D}_{\text{train}}}$ for the fit to all 30 observations, though (Remark 0.9). Each of its fits uses only 24 observations. At degree 10, polynomials fitted to 24 points predict their held-out points much worse than the fit to all 30 predicts new data: the cross-validation estimate is 4.1 times the estimated generalization RMSE.

Part of that gap is expected: by Remark 0.9, with g the least-squares fit of a degree-10 polynomial, the cross-validation estimate targets $\mathrm{Err}^{g}(24)$, not the error of a fit to 30 points. A simulation of many training sets of each size shows how much the size alone matters at this degree.

```r
sim_generalization_draws <- function(
  n_train, degree, n_reps, draw, new_data, score
) {
  purrr::map_dbl(seq_len(n_reps), \(r) {
    fit <- lm(y ~ poly(x, degree), data = draw(n_train))
    score(fit, new_data)
  })
}

sim_n_reps <- 500
set.seed(24)
sim_size_effect <- tibble::tibble(
  n_train = c(sim_n_fit, nrow(sim_train))
) |>
  dplyr::mutate(
    draws = purrr::map(
      n_train,
      \(n) {
        sim_generalization_draws(
          n, max(sim_degrees), sim_n_reps,
          draw = sim_draw, new_data = sim_new, score = prediction_mse
        )
      }
    ),
    median = purrr::map_dbl(draws, stats::median),
    quantile_90 = purrr::map_dbl(draws, \(v) stats::quantile(v, 0.9)),
    mean = purrr::map_dbl(draws, mean),
    largest = purrr::map_dbl(draws, max),
    top_two_share = purrr::map_dbl(
      draws,
      \(v) sum(sort(v, decreasing = TRUE)[1:2]) / sum(v)
    )
  ) |>
  dplyr::select(-draws)

sim_size_effect
#> # A tibble: 2 x 6
#>   n_train median quantile_90   mean   largest top_two_share
#>     <int>  <dbl>       <dbl>  <dbl>     <dbl>         <dbl>
#> 1      24  1.14        207.  39068. 18318345.         0.965
#> 2      30  0.274        40.6   116.    24568.         0.508
```

Over 500 simulated training sets of each size, degree-10 fits to 24 points have median generalization MSE 1.14, against 0.27 for fits to 30 points, and 90th percentiles of 207.31 and 40.65. The sample means are no guide to $\mathrm{Err}^g(24)$ here: the largest single value among the fits to 24 points is 18,000,000, from one fit that is wildly wrong away from the data, and the two largest values supply 97% and 51% of the two means. So fits of this degree to 24 points are much worse than fits to 30 points, and a large cross-validation estimate here is no surprise. How much of this particular gap comes from the training-set size, and how much from the chance of this one data set, a single run cannot say. The estimate also jumps around from one degree to the next, because it rests on only 30 observations rather than on 10,000 new ones.

Example (Four-fold cross-validation of the `mtcars` polynomials)

Example 0.9 (Four-fold cross-validation of the `mtcars` polynomials)**.** This example applies the $k = 4$ folds of Exercise 0.7 to the four polynomial models of Exercise 0.1, now using all $n = 32$ cars and the `cv_mse()` function of Example 0.8. The fold assignment is random, so the estimate is too; to show how much, we repeat the whole procedure with 5 different random fold assignments.

Table 4: Square root of the 4-fold cross-validation estimate, by degree, for each of 5 random fold assignments

```
mtcars_degrees <- 1:4
n_repeats <- 5

set.seed(32)
mtcars_cv <-
  tidyr::expand_grid(repeat_id = seq_len(n_repeats), degree = mtcars_degrees) |>
  dplyr::mutate(
    rmse = {
      folds <- assign_folds(nrow(mtcars), k = 4)
      purrr::map_dbl(
        degree,
        \(d) sqrt(cv_mse(mpg ~ poly(wt, d), mtcars, folds))
      )
    },
    .by = repeat_id
  ) |>
  tidyr::pivot_wider(
    names_from = degree,
    values_from = rmse,
    names_prefix = "degree_"
  )

mtcars_cv
#> # A tibble: 5 x 5
#>   repeat_id degree_1 degree_2 degree_3 degree_4
#>       <int>    <dbl>    <dbl>    <dbl>    <dbl>
#> 1         1     3.42     2.67     2.69     2.92
#> 2         2     3.10     2.73     2.74     2.82
#> 3         3     3.14     2.82     2.86     3.00
#> 4         4     3.18     2.65     2.76     3.00
#> 5         5     3.38     2.74     2.78     2.81
```

Table 5: Degree with the smallest 4-fold cross-validated RMSE in each of 5 random fold assignments

```
mtcars_cv_best <-
  mtcars_cv |>
  tidyr::pivot_longer(
    -repeat_id,
    names_to = "degree",
    names_prefix = "degree_",
    values_to = "rmse"
  ) |>
  dplyr::slice_min(rmse, n = 1, by = repeat_id)

mtcars_cv_best
#> # A tibble: 5 x 3
#>   repeat_id degree  rmse
#>       <int> <chr>  <dbl>
#> 1         1 2       2.67
#> 2         2 2       2.73
#> 3         3 2       2.82
#> 4         4 2       2.65
#> 5         5 2       2.74
```

```
mtcars_cv_range <-
  mtcars_cv |>
  dplyr::summarise(
    dplyr::across(dplyr::starts_with("degree_"), \(v) diff(range(v)))
  )
mtcars_chosen <- sort(unique(mtcars_cv_best$degree))
mtcars_choice_text <- if (length(mtcars_chosen) == 1) {
  paste0(
    "but every assignment chose degree ", mtcars_chosen,
    ": the estimates moved, and the winner did not"
  )
} else {
  paste0(
    "and the chosen degree was ",
    knitr::combine_words(mtcars_chosen, and = " or "),
    ", depending on how the cars happened to be split"
  )
}
mtcars_cv_range
#> # A tibble: 1 x 4
#>   degree_1 degree_2 degree_3 degree_4
#>      <dbl>    <dbl>    <dbl>    <dbl>
#> 1    0.320    0.168    0.164    0.195
```

Each row of Table 4 holds, for each degree, the square root of $\widehat{\mathrm{Err}}_{\mathrm{CV}(4)}$ (Equation 28) for one fold assignment. Across fold assignments, the estimate for one degree varied by as much as 0.32 mpg, but every assignment chose degree 2: the estimates moved, and the winner did not. A single run of k-fold cross-validation reports one of these rows, so a small difference between two models' estimates may not survive a different split.

Leave-one-out cross-validation

Exercise (The largest number of folds)

Exercise 0.9 (The largest number of folds)**.** Suppose we take $k = n$ in k-fold cross-validation on the $n = 32$ cars in `mtcars`.

1. How many cars are in each fold?
2. How many models are fit, and how many cars is each fitted to?
3. If we run the procedure twice, with two different random fold assignments, can the

two values of $\widehat{\mathrm{Err}}_{\mathrm{CV}(32)}$ (Equation 28) differ?

Solution

Solution 0.9.

1. $32/32 = 1$ car per fold.
2. One model per fold, so 32 models, each fitted to the $32 - 1 = 31$ other cars.
3. No. With one car per fold, every assignment of cars to folds produces the same 32 fits, only numbered differently, and Equation 28 does not depend on the numbering. Unlike Example 0.9, the estimate is not random.

Definition (Leave-one-out cross-validation)

Definition 0.31 (Leave-one-out cross-validation)**. Leave-one-out cross-validation (LOOCV)** is k-fold cross-validation with $k = n$, so that each of the n folds holds one observation, and the fold-out rule $\hat{y}^{(-\kappa(i))}(\cdot)$ of observation i's fold is fitted to all observations except observation i. Its cross-validation estimate (Equation 28 with $k = n$) is

$$\widehat{\mathrm{Err}}_{\mathrm{CV}(n)} = \frac{1}{n}\sum_{i \in I}\left(\hat{y}^{(-\kappa(i))}(x_i) - y_i\right)^2. \tag{31}$$

Source

James et al. (2021, sec. 5.1.2).

Definition (Term vector)

Definition 0.32 (Term vector)**.** Consider a model whose mean is a linear combination of p^* terms $t_1(x), \ldots, t_{p^*}(x)$ of the covariate value, such as 1, x and x^2. The **term vector** $\tilde{t}_i$ of observation i is the column of its terms:

$$\tilde{t}_i \stackrel{\text{def}}{=} \left(t_1(x_i), \ldots, t_{p^*}(x_i)\right)^\top. \tag{32}$$

Remark (The term vector of simple linear regression)

Remark 0.11 (The term vector of simple linear regression). In simple linear regression the terms are 1 and x, so the term vector $\tilde{t}_i$ is the covariate vector $\tilde{x}_i$.

Definition (Cross-product matrix)

Definition 0.33 (Cross-product matrix)**.** The **cross-product matrix** A of term vectors $\tilde{t}_i$, $i \in I$, of a training set indexed by I, is the sum of their outer products:

$$A \stackrel{\text{def}}{=} \sum_{i \in I} \tilde{t}_i \tilde{t}_i^\top . \tag{33}$$

Source

Sanchez (2020, chap. "Matrix Cross-Products") (see the chapter online) names this matrix a sum-of-squares and cross-products matrix.

Remark (Cross-product matrix as a matrix product)

Remark 0.12 (Cross-product matrix as a matrix product). Let D be the matrix whose rows are the transposed term vectors $\tilde{t}_i^\top$, $i \in I$. The cross-product matrix then equals the matrix product

$$A = D^\top D.$$

Its diagonal entries are sums of squares of one term over the training set. Its off-diagonal entries are sums of cross-products of two different terms. Statisticians therefore call A the sum-of-squares and cross-products (SSCP) matrix of the terms. When every column of D has mean zero, $A/(|I| - 1)$ is the sample covariance matrix of the terms.

Source

I restated the SSCP reading and the covariance special case from Sanchez (2020, chap. "Matrix Cross-Products") (see the chapter online) in this page's notation. The book carries a non-commercial, share-alike license, so I paraphrased it and copied none of its text.

Definition (Leverage)

Definition 0.34 (Leverage)**.** Consider a linear model fitted by ordinary least squares, with term vectors $\tilde{t}_i$, $i \in I$, and an invertible cross-product matrix A. The **leverage** of observation i is

$$h_i \stackrel{\text{def}}{=} \tilde{t}_i^\top A^{-1} \tilde{t}_i. \tag{34}$$

Source

James et al. (2021, sec. 3.3.3), written here with term vectors and the matrix

A, as in the vector form of the OLS estimates.

Theorem (Leave-one-out cross-validation for least squares)

Theorem 0.5 (Leave-one-out cross-validation for least squares)**.** *For a linear model fitted by ordinary least squares with every leverage $h_i < 1$, the leave-one-out estimate can be computed from the single fit to all n observations, from its fitted values $\hat{y}_i$, their prediction errors $\varepsilon_i = \hat{y}_i - y_i$, and the leverages:*

$$\widehat{\mathrm{Err}}_{CV(n)} = \frac{1}{n} \sum_{i \in I} \left(\frac{\varepsilon_i}{1 - h_i} \right)^2 . \tag{35}$$

Source

James et al. (2021, sec. 5.1.2) states this formula for least squares linear or polynomial regression, and Hastie et al. (2009, sec. 7.10.1) for many linear fitting methods.

Proof

Proof. Not derived here; see the sources named under the theorem. The derivation needs a formula for how the inverse of the cross-product matrix A changes when one observation's outer product $\tilde{t}_i \tilde{t}_i^\top$ is removed from it (the Sherman–Morrison formula), which this page does not develop. □

Example (Leave-one-out cross-validation of the `mtcars` polynomials)

Example 0.10 (Leave-one-out cross-validation of the `mtcars` polynomials)**.** This example computes $\widehat{\mathrm{Err}}_{\mathrm{CV}(32)}$ for the four `mtcars` polynomials of Exercise 0.1 in two ways: by fitting each model 32 times, as in Equation 31, and by the single-fit formula of Theorem 0.5, with the leverages from `hatvalues()`.

```r
loocv_by_formula <- function(formula, data) {
  fit <- lm(formula, data = data)
  pred_error <- fitted(fit) - stats::model.response(model.frame(fit))
  mean((pred_error / (1 - hatvalues(fit)))^2)
}

mtcars_loocv <- tibble::tibble(degree = mtcars_degrees) |>
  dplyr::mutate(
    refitting = purrr::map_dbl(
      degree,
      \(d) cv_mse(mpg ~ poly(wt, d), mtcars, folds = seq_len(nrow(mtcars)))
    ),
    formula = purrr::map_dbl(
      degree,
      \(d) loocv_by_formula(mpg ~ poly(wt, d), mtcars)
    ),
    loocv_rmse = sqrt(formula)
  )

mtcars_loocv
#> # A tibble: 4 x 4
#>   degree refitting formula loocv_rmse
#>    <int>     <dbl>   <dbl>      <dbl>
#> 1      1     10.3    10.3        3.20
#> 2      2      7.79    7.79       2.79
#> 3      3      8.34    8.34       2.89
#> 4      4      8.71    8.71       2.95
```

```r
loocv_max_diff <- max(abs(mtcars_loocv$refitting - mtcars_loocv$formula))
mtcars_best_loocv <-
  mtcars_loocv$degree[which.min(mtcars_loocv$formula)]
c(max_difference = loocv_max_diff, best_degree = mtcars_best_loocv)
#> max_difference    best_degree
#>    8.88178e-15    2.00000e+00
```

The two columns differ by at most 8.9e-15, which is rounding error. The single-fit formula needs one fit per model instead of 32. LOOCV picks degree 2, and because LOOCV is not random (Solution 0.9), rerunning it always gives the same choice.

Remark (When to use leave-one-out cross-validation)

Remark 0.13 (When to use leave-one-out cross-validation). LOOCV is the $k = n$ end of Remark 0.10. By Remark 0.9, $\widehat{\mathrm{Err}}_{\mathrm{CV}(n)}$ estimates $\mathrm{Err}^g(n-1)$ for the fitting procedure g: each rule is fitted to $n-1$ observations, so the bias of $\widehat{\mathrm{Err}}_{\mathrm{CV}(n)}$ as an estimate of $\mathrm{Err}^g(n)$ is the smallest of any k. But the n training sets are nearly identical, so the variance of $\widehat{\mathrm{Err}}_{\mathrm{CV}(n)}$ can be larger than that of 5- or 10-fold cross-validation (James et al. 2021, sec. 5.1.4). For least squares fits, Theorem 0.5 computes LOOCV from a single fit; for most other fitting methods, LOOCV requires n fits.

Choosing a model by cross-validation

Definition (Cross-validation estimate of a named procedure)

Definition 0.35 (Cross-validation estimate of a named procedure)**.** Let a training set $\mathcal{D}_{\mathrm{train}}$, indexed by I, be divided into folds $\mathcal{F}_1, \dots, \mathcal{F}_k$. The **cross-validation estimate of procedure** g on $\mathcal{D}_{\mathrm{train}}$, $\widehat{\mathrm{Err}}^g_{\mathrm{CV}(k)}(\mathcal{D}_{\mathrm{train}})$, is the cross-validation estimate computed with fold-out rules that g produces from $\mathcal{D}_{\mathrm{train}}$:

$$\widehat{\mathrm{Err}}^g_{\mathrm{CV}(k)}(\mathcal{D}_{\mathrm{train}}) \stackrel{\mathrm{def}}{=} \frac{1}{n}\sum_{i\in I}\Big(g\big(x_i; ((x_{i'}, y_{i'}))_{i'\in I\setminus\mathcal{F}_{\kappa(i)}}\big) - y_i\Big)^2. \tag{36}$$

Definition (Cross-validation choice procedure)

Definition 0.36 (Cross-validation choice procedure)**.** Given candidate fitting procedures $g_1, \dots, g_L$, the **cross-validation choice procedure** g^{CV} is the fitting procedure that, applied to a training set $\mathcal{D}_{\mathrm{train}}$, divides $\mathcal{D}_{\mathrm{train}}$ into k folds, computes each candidate's cross-validation estimate on $\mathcal{D}_{\mathrm{train}}$ with those folds, and applies to $\mathcal{D}_{\mathrm{train}}$ the candidate with the smallest estimate, taking the smallest l if several tie:

$$g^{\mathrm{CV}}(x; \mathcal{D}_{\mathrm{train}}) \stackrel{\mathrm{def}}{=} g_{\min\arg\min_l \widehat{\mathrm{Err}}^{g_l}_{\mathrm{CV}(k)}(\mathcal{D}_{\mathrm{train}})}(x; \mathcal{D}_{\mathrm{train}}). \tag{37}$$

Definition (Nested cross-validation)

Definition 0.37 (Nested cross-validation)**. Nested cross-validation** is k-fold cross-validation of the cross-validation choice procedure g^{CV}: its estimate on a training set $\mathcal{D}_{\mathrm{train}}$ is

$$\widehat{\mathrm{Err}}^{g^{\mathrm{CV}}}_{\mathrm{CV}(k)}(\mathcal{D}_{\mathrm{train}}). \tag{38}$$

Source

Hastie et al. (2009, sec. 7.10.2) for cross-validating every step of a procedure, choices included.

Remark (What is nested)

Remark 0.14 (What is nested). Each fold-out rule of g^{CV} is computed from the observations outside one outer fold $\mathcal{F}_j$, so the whole choice, including the cross-validation that g^{CV} runs inside those observations, never sees $\mathcal{F}_j$. That inner cross-validation is the nested one.

Remark (Cross-validation replaces the validation set, not the test set)

Remark 0.15 (Cross-validation replaces the validation set, not the test set). Used to choose between models, cross-validation takes the place of the *validation* set of Definition 0.19, the one consulted again and again, and not of the test set, which stays unused until the end. The cross-validation estimate of the winning model is optimistic: that model won partly because the particular sample and split happened to favor it, so its estimate is the smallest of several noisy estimates. An honest estimate of the chosen model's error needs data that took no part in the choice (Hastie et al. 2009, sec. 7.2), such as a test set, or the outer folds of nested cross-validation.

Source

Based on a Spring 2025 lecture on generalization by Logan Sizemore.

Definition (Fold-based standard error)

Definition 0.38 (Fold-based standard error)**.** The **fold-based standard error** $\mathrm{SE}_{\mathrm{CV}(k)}$ of a k-fold cross-validation estimate is the sample standard deviation of the k fold mean squared errors, divided by $\sqrt{k}$:

$$\mathrm{SE}_{\mathrm{CV}(k)} \stackrel{\mathrm{def}}{=} \frac{\widehat{\mathrm{SD}}(\mathrm{MSE}_1, \ldots, \mathrm{MSE}_k)}{\sqrt{k}}. \tag{39}$$

Source

Hastie et al. (2009, sec. 7.10.1), who call the result a standard error.

Remark (The fold-based standard error is only a rough guide)

Remark 0.16 (The fold-based standard error is only a rough guide). Hastie et al. (2009, sec. 7.10.1) draw standard-error bars computed from the per-fold errors as in Equation 39. That formula treats the k fold errors as if they were independent, which they are not: every pair of folds shares $k-2$ folds of training data. So $\mathrm{SE}_{\mathrm{CV}(k)}$ is a rough indication of how much $\widehat{\mathrm{Err}}_{\mathrm{CV}(k)}$ might change with a different sample, not a standard error of $\widehat{\mathrm{Err}}_{\mathrm{CV}(k)}$.

Definition (Fold mean squared error of a named procedure)

Definition 0.39 (Fold mean squared error of a named procedure)**.** The **fold mean squared error of procedure** g for fold $\mathcal{F}_j$ of a training set $\mathcal{D}_{\text{train}}$, indexed by I, is the fold mean squared error computed with the fold-out rule that g produces:

$$\mathrm{MSE}_j^g(\mathcal{D}_{\text{train}}) \stackrel{\text{def}}{=} \frac{1}{n_j} \sum_{i \in \mathcal{F}_j} \Big(g\big(x_i; ((x_{i'}, y_{i'}))_{i' \in I \setminus \mathcal{F}_j}\big) - y_i \Big)^2. \tag{40}$$

Definition (Fold-based standard error of a named procedure)

Definition 0.40 (Fold-based standard error of a named procedure)**.** The **fold-based standard error of procedure** g on a training set $\mathcal{D}_{\text{train}}$, $\mathrm{SE}^g_{\mathrm{CV}(k)}(\mathcal{D}_{\text{train}})$, is the fold-based standard error computed from the fold mean squared errors of g:

$$\mathrm{SE}^g_{\mathrm{CV}(k)}(\mathcal{D}_{\text{train}}) \stackrel{\text{def}}{=} \frac{\widehat{\mathrm{SD}}(\mathrm{MSE}_1^g(\mathcal{D}_{\text{train}}), \dots, \mathrm{MSE}_k^g(\mathcal{D}_{\text{train}}))}{\sqrt{k}}. \tag{41}$$

Definition (One-standard-error rule)

Definition 0.41 (One-standard-error rule)**.** Let $g_1, \dots, g_L$ be candidate fitting procedures, each at least as flexible as the one before, cross-validated on a training set $\mathcal{D}_{\text{train}}$ with the same folds, and let g_{l^*} be the one with the smallest cross-validation estimate. The **one-standard-error rule** chooses the smallest l whose estimate is within one fold-based standard error of the smallest:

$$\widehat{\mathrm{Err}}^{g_l}_{\mathrm{CV}(k)}(\mathcal{D}_{\text{train}}) \le \widehat{\mathrm{Err}}^{g_{l^*}}_{\mathrm{CV}(k)}(\mathcal{D}_{\text{train}}) + \mathrm{SE}^{g_{l^*}}_{\mathrm{CV}(k)}(\mathcal{D}_{\text{train}}). \tag{42}$$

Source

Hastie et al. (2009, sec. 7.10.1); James et al. (2021, sec. 6.1.3).

Example (The one-standard-error rule for the simulated polynomials)

Example 0.11 (The one-standard-error rule for the simulated polynomials)**.** This example applies 10-fold cross-validation to the simulated training set of Example 0.4, records the mean squared error in each fold, and applies Definition 0.41 to the ten degrees.

```r
set.seed(10)
sim_folds_ten <- assign_folds(nrow(sim_train), k = 10)

sim_cv_ten <- tibble::tibble(degree = sim_degrees) |>
  dplyr::mutate(
    fold_mse = purrr::map(
      degree,
      \(d) {
        sq_errors <- cv_squared_errors(y ~ poly(x, d), sim_train, sim_folds_ten)
        as.numeric(tapply(sq_errors, sim_folds_ten, mean))
      }
    ),
    cv = purrr::map_dbl(fold_mse, mean),
    se = purrr::map_dbl(fold_mse, \(v) sd(v) / sqrt(length(v)))
  ) |>
  dplyr::select(-fold_mse)

sim_min <- dplyr::slice_min(sim_cv_ten, cv, n = 1)
sim_one_se <-
  sim_cv_ten |>
  dplyr::filter(cv <= sim_min$cv + sim_min$se) |>
  dplyr::slice_min(degree, n = 1)

sim_cv_ten
#> # A tibble: 10 x 3
#>    degree     cv     se
#>     <int>  <dbl>  <dbl>
#>  1      1 0.366  0.0962
#>  2      2 0.447  0.127
#>  3      3 0.0714 0.0219
#>  4      4 0.0846 0.0235
#>  5      5 0.0919 0.0244
#>  6      6 0.0792 0.0214
#>  7      7 0.202  0.104
#>  8      8 0.0619 0.0161
#>  9      9 0.247  0.134
#> 10     10 0.150  0.0611
```

With 30 observations and 10 folds, each fold holds 3 observations, so all ten folds have equal size and `cv` is $\widehat{\mathrm{Err}}_{\mathrm{CV}(10)}$ (Remark 0.9).

```
sim_cv_ten |>
  ggplot2::ggplot(ggplot2::aes(degree, cv)) +
  ggplot2::geom_hline(
    yintercept = sim_min$cv + sim_min$se,
    linetype = "dashed"
  ) +
  ggplot2::geom_errorbar(
    ggplot2::aes(ymin = cv - se, ymax = cv + se),
    width = 0.2
  ) +
  ggplot2::geom_point(size = 2) +
  ggplot2::scale_x_continuous(breaks = sim_degrees) +
  ggplot2::scale_y_log10() +
  ggplot2::labs(
    x = "polynomial degree",
    y = "10-fold CV estimate (log scale)"
  ) +
  ggplot2::theme_bw()
```

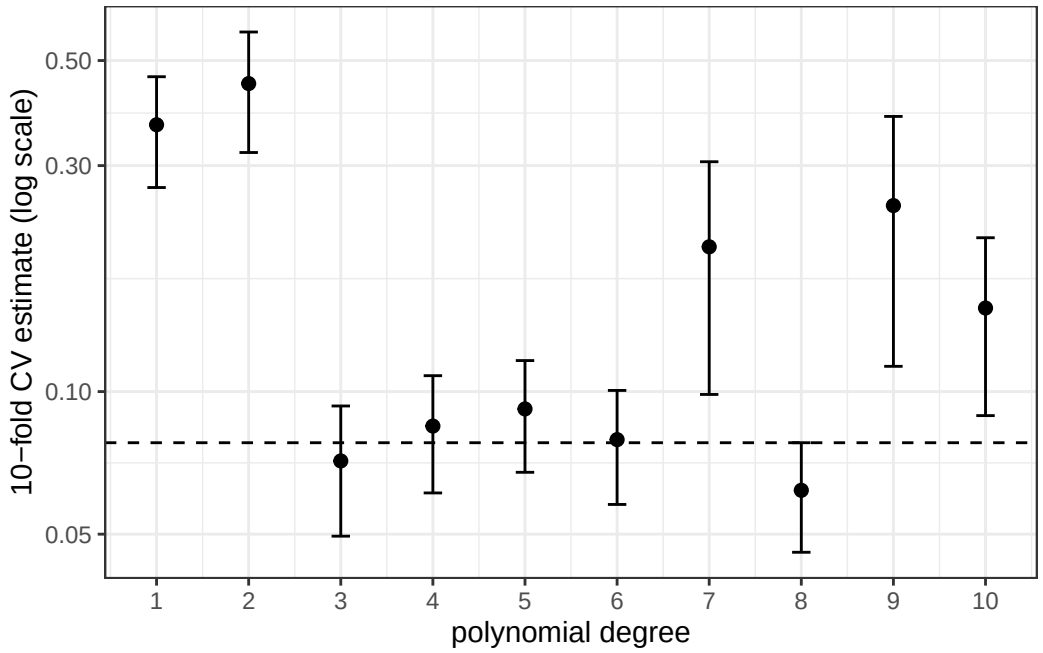


Figure 7: 10-fold cross-validation estimates, with bars of one fold-based standard error (Equation 39). The dashed line is one fold-based standard error above the smallest estimate.

The smallest estimate is at degree 8. The lowest degree within one fold-based standard error of it is degree 3, which is the one-standard-error rule's choice. The rule prefers the simpler model when the data cannot clearly tell it apart from the degree with the smallest estimate. Because the data are simulated, we can check both choices against the estimated generalization RMSE of Example 0.4 for each: 0.335 for degree 8 and 0.315 for degree 3.

Cross-validating the whole procedure

Exercise (Screening predictors before cross-validating)

Exercise 0.10 (Screening predictors before cross-validating)**.** A data set has $n = 50$ observations of an outcome Y and of $p = 1000$ candidate predictors. In fact every variable is independent standard normal noise, so no predictor carries any information about Y. An analyst

1. picks the 10 predictors most correlated with Y in all 50 observations, then
2. estimates the prediction error of a linear model in those 10 predictors by 5-fold cross-validation.

Answer the following:

1. What is the smallest possible generalization error (Equation 7) of any prediction rule for Y here?
2. Simulate the analyst's procedure. What cross-validation estimate does it report?
3. Repeat the simulation, but carry out step 1 inside each fold, using only that fold's training observations. What estimate does this give?

Source

A regression version of the classification example in Hastie et al. (2009, sec. 7.10.2), "The Wrong and Right Way to Do Cross-validation".

Solution

Solution 0.10.

1. Y is independent of every predictor, with mean 0 and variance 1. The best prediction is the constant 0, and its generalization error is $\mathrm{Var}(Y) = 1$. Every other rule does worse, so any honest estimate should be near or above 1.

2. and 3. The function below runs one data set through both procedures. The only

> difference is whether the screening step sees the whole data set (`"outside"`) or only the training folds (`"inside"`).

```r
top_predictors <- function(x, y, n_keep) {
  order(abs(stats::cor(x, y)), decreasing = TRUE)[seq_len(n_keep)]
}

screened_cv <- function(x, y, folds, n_keep, screen) {
  if (screen == "outside") {
    keep <- top_predictors(x, y, n_keep)
  }
  sq_errors <- numeric(length(y))
  for (j in unique(folds)) {
    in_fold <- folds == j
    if (screen == "inside") {
      keep <- top_predictors(x[!in_fold, ], y[!in_fold], n_keep)
    }
    train <- tibble::tibble(
      y = y[!in_fold],
      tibble::as_tibble(x[!in_fold, keep])
    )
    fit <- lm(y ~ ., data = train)
    held_out <- tibble::as_tibble(x[in_fold, keep])
    sq_errors[in_fold] <- (predict(fit, newdata = held_out) - y[in_fold])^2
  }
  mean(sq_errors)
}

set.seed(7102)
n_obs <- 50
n_predictors <- 1000
n_sims <- 50

screening_results <- purrr::map(seq_len(n_sims), \(s) {
  x <- matrix(stats::rnorm(n_obs * n_predictors), n_obs, n_predictors)
  colnames(x) <- paste0("x", seq_len(n_predictors))
  y <- stats::rnorm(n_obs)
  folds <- assign_folds(n_obs, k = 5)
  tibble::tibble(
    sim = s,
    outside = screened_cv(x, y, folds, n_keep = 10, screen = "outside"),
    inside = screened_cv(x, y, folds, n_keep = 10, screen = "inside")
  )
}) |>
  purrr::list_rbind()

screening_summary <-
  screening_results |>
  dplyr::summarise(
    dplyr::across(c(outside, inside), mean)
  )

screening_summary
#> # A tibble: 1 x 2
#>   outside inside
```

Averaged over 50 simulated data sets:

- Screening outside the folds gives a mean CV estimate of 0.54, well below the smallest possible generalization error of 1. The procedure reports that pure noise predicts Y.
- Screening inside the folds gives 1.52, above 1, as an honest estimate for a rule built from noise should be.

The 10 predictors chosen from all 50 observations were chosen *because* they happen to correlate with Y in the very observations that later serve as held-out folds. Those observations were not held out from the whole fitting procedure, so the independence that Theorem 0.3 needs fails.

Remark (Cross-validate the whole procedure)

Remark 0.17 (Cross-validate the whole procedure). Every step that uses the outcome is part of fitting, and must be repeated inside each fold using only that fold's training observations (Hastie et al. 2009, sec. 7.10.2). Such steps include:

- screening or selecting predictors, as in Exercise 0.10;
- choosing transformations or a polynomial degree;
- choosing a penalty or another tuning parameter.

Hastie et al. (2009, sec. 7.10.2) note that steps using only the predictors, such as dropping predictors with almost no variation, can be done once on all the data before the folds are formed. Repeating such a step inside each fold is never wrong, though, and it scores each held-out fold the way new data would be scored. Examples are estimating the mean and standard deviation used for standardization, and the values used to fill in missing predictor values, when they are computed without the outcome.

Further reading

Sanchez (2020) reviews the matrix algebra behind statistical learning, including the covariance and correlation matrices that cross-products produce.

References

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer. https://doi.org/10.1007/978-0-387-84858-7.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed. Springer. https://doi.org/10.1007/978-1-0716-1418-1.

Sanchez, Gaston. 2020. *A Matrix Algebra Companion for Statistical Learning (matrix4sl)*. Online book, work in progress. Licensed CC BY-NC-SA 4.0. https://www.gastonsanchez.com/matrix4sl/.